National Dong Hwa University

Department of Computer Science and Information Engineering (CSIE)

CSIE Undergraduate Project

AI-Driven Network and IoT Device Management Using Agentic Automation Workflows

Final Report

Group Members

Ayanda Sibusiso Dlamini	411221337
Thubelihle Bruno Maseko	411221351
Mlandvo Welcome Nkambule	411221332
Luhlelo Bonkhe Dube	411221323

Supervisor: Prof. Hung-Jui Chang

University: National Dong Hwa University (NDHU)

Department: Department of Computer Science and Information Engineering (CSIE)

Date: March 2026

Abstract

This report presents the academic and technical foundation for the project titled *AI-Driven Network and IoT Device Management Using Agentic Automation Workflows*. Chapter 1 introduces the problem context, project motivation, aims, significance, scope, and the overall structure of the study, based on the source project report provided by the project team. Chapter 2 consolidates the developed literature review and examines traditional network management, network automation, software-defined and intent-based approaches, artificial intelligence in network operations, large language model (LLM)-enabled assistance, agentic workflow orchestration, and AI-driven IoT management.

The report argues that current network and IoT administration remains operationally complex because conventional approaches depend heavily on manual configuration, static automation logic, and specialist technical knowledge. The literature indicates that AI-assisted systems can reduce this barrier by supporting natural-language interaction, contextual diagnostics, and intelligent task decomposition. However, it also shows that safe execution remains essential; therefore, deterministic automation, workflow validation, and human oversight are still necessary design requirements.

The combined evidence supports a hybrid architecture in which agentic AI assists with intent interpretation, planning, reporting, and diagnostics, while workflow engines and automation tooling provide controlled, auditable execution. The final evaluation demonstrates this approach across systems administration, systems security monitoring, employee network issue reporting, and administrator network operations.

Table of Contents

Abstract i

List of Figures vi

List of Tables vii

List of Abbreviations viii

1 Introduction 1

1.1 Background of the Study . . . 1
1.2 Statement of the Problem . . . 2
1.3 Aim and Objectives . . . 3
1.4 Significance of the Study . . . 3
1.5 Scope and Limitations . . . 4
1.6 Structure of the Report . . . 5

2 Literature Review 6

2.1 Traditional Network Management Approaches . . . 6
2.2 Network Automation and Programmability . . . 6
2.3 Intent-Based Networking . . . 7
2.4 Artificial Intelligence in Network Management . . . 7
2.5 Large Language Models in Network Operations . . . 8
2.6 Agentic Systems and Workflow Orchestration . . . 8
2.7 AI-Driven IoT Network Management . . . 9
2.8 Research Gaps . . . 9

Review Matrix 11

3 System Architecture and Design 14

3.1 Overall System Architecture . . . 14
3.1.1 Virtualized Infrastructure Environment . . . 14
3.1.2 Virtual Machine Architecture . . . 15
3.1.3 Layered Interaction Model . . . 17
3.1.4 System Operation Flow . . . 18
3.1.5 Operational Safety and Human Oversight . . . 19
3.1.6 Architectural Summary . . . 19
3.2 Design Principles . . . 19

3.3 Agentic Workflow Design . . . 21
3.3.1 Role of the AI Agent . . . 21
3.3.2 Agent Reasoning Model (Observe–Think–Decide–Act) . . . 21
3.4 Automation Orchestration Layer . . . 23
3.4.1 The Orchestration Bridge . . . 24
3.4.2 Separation of Reasoning and Execution . . . 24
3.4.3 Workflow Execution Model . . . 24

4 System Implementation **26**
4.1 Development Environment . . . 26
4.1.1 Virtualization Platform . . . 26
4.1.2 Simulation Tools . . . 27
4.1.3 Operating Systems . . . 28
4.2 Network Testbed Setup . . . 28
4.2.1 Enterprise Network Topology . . . 28
4.2.2 VLANs and Segmentation . . . 29
4.2.3 Routing and Firewall Configuration . . . 29
4.3 AI and Agent Implementation . . . 30
4.3.1 Model Selection . . . 30
4.3.2 Prompting and Intent Extraction . . . 31
4.4 Workflow Automation Implementation . . . 31
4.4.1 Workflow Definitions . . . 32
4.4.2 Integration with Automation Scripts . . . 32
4.5 Python Automation Libraries . . . 33
4.5.1 Core Prototype Device Access . . . 33
4.5.2 Extended Monitoring and Deployment Support . . . 34
4.6 Security and Access Control . . . 34
4.6.1 Prototype-Level Security Controls . . . 35
4.6.2 Security Considerations for Real-World Deployment . . . 36

5 Experimental Evaluation and Results **39**
5.1 Systems Layer Evaluation . . . 39
5.1.1 Systems Infrastructure Administration Evaluation . . . 39
5.1.2 Systems Security Monitoring Evaluation . . . 55
5.2 Network Layer Evaluation . . . 61
5.2.1 Employee Network Issue Reporting and Policy Validation . . . 61
5.2.2 Admin Network Operations Bot Evaluation . . . 67

6 Discussion **74**
6.1 Discussion Overview . . . 74
6.2 Interpretation of Findings . . . 74
6.3 Human-in-the-Loop Control . . . 75

6.4 Operational Benefits 75
6.5 Limitations 75
6.6 Lessons Learned 76

7 Conclusion and Future Work **77**
7.1 Conclusion 77
7.2 Project Contributions 77
7.3 Future Work 78
7.4 Final Remarks 78

List of Figures

3.1 Virtualized system architecture showing the Proxmox host and deployed virtual machines. . . . 16
3.2 System administration layer interaction model showing the flow from user request to Proxmox-based company system simulation. . . . 17
3.3 Network layer interaction model showing the layered process from user input to GNS3-based network device management. . . . 18
3.4 AI agent reasoning cycle showing the Observe–Think–Decide–Act process, human-in-the-loop approval, n8n decision handling, and execution feedback. 22

4.1 Implementation architecture showing the virtualized Proxmox host, the four deployed virtual machines, and the subcomponents hosted within each VM (n8n, Docker, Portainer, LLMs, and GNS3 Client). . . . 27
4.2 System interaction flow showing the layered pipeline from user request through the chat interface, AI agent, n8n workflow engine, and Python automation layer to the GNS3 simulated network environment. . . . 33

5.1 Evaluation environment and Proxmox Admin Bot workflow overview. . . 40
5.2 OpenClaw Proxmox Orchestrator sub-workflow overview. . . . 41
5.3 AI-assisted systems infrastructure administration workflow. . . . 44
5.4 Telegram message intake and main bot communication. . . . 44
5.5 Safe patching request, confirmation prompt, administrator approval, snapshot creation, and health-check result. . . . 46
5.6 Formatted VM IP address summary. . . . 48
5.7 Slow VM diagnostic request, Proxmox metrics, guest OS snapshot, and recommendation output. . . . 50
5.8 Unreachable VM recovery request, VM state, IP resolution, and least-disruptive recovery recommendation. . . . 51
5.9 Idle VM cleanup request and recommendation-only cleanup output. . . . 52
5.10 AI Security Guard workflow design. . . . 57
5.11 Telegram command examples and generated AI Security Guard report for the last 1 hour. . . . 58
5.12 Intern user triggering repeated company security warnings and the resulting policy violation summary. . . . 59

5.13 Scheduled threshold alert and timeframe metadata evidence from the AI Security Guard workflow. 60
5.14 Employee Network Case Logging workflow design in n8n. 62
5.15 General operational flow of the Employee Network Issue Reporting and Policy Validation workflow. 63
5.16 Employee-side policy validation for blocked YouTube access. 64
5.17 Employee-side ACL validation for Guest network access to the HR department. 65
5.18 Employee-side case logging for an unresolved internal department access issue. 65
5.19 Employee-side escalation and case logging for an internal server access issue. 66
5.20 General operational flow of the Admin Network Operations Bot workflow. 68
5.21 Admin Network Operations Bot help output. 69
5.22 Administrator case inspection through Telegram. 70
5.23 Case status update, resolution confirmation, and employee notification. . 70
5.24 Administrator diagnostic plan generated for a network support case. 71
5.25 VLAN visibility and VLAN creation results. 71
5.26 Approval-controlled VLAN deletion request and rejection result. 72
5.27 Denied firewall log retrieval output. 73
5.28 Network administration data tables used for support records, approval state, and diagnostic history. 73

List of Tables

2.1 Review matrix of the principal sources used in the literature review. 11

5.1 Systems infrastructure administration evaluation environment. 40

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CSIE	Computer Science and Information Engineering
EIGRP	Enhanced Interior Gateway Routing Protocol
GNS3	Graphical Network Simulator-3
IBN	Intent-Based Networking
IoT	Internet of Things
LLM	Large Language Model
MDT	Model-Driven Telemetry
NDHU	National Dong Hwa University
RAG	Retrieval-Augmented Generation
SDN	Software-Defined Networking
SNMP	Simple Network Management Protocol
SSH	Secure Shell
TLS	Transport Layer Security
YANG	Yet Another Next Generation

Chapter 1

Introduction

1.1 Background of the Study

Modern digital infrastructure relies heavily on interconnected networks composed of routers, switches, servers, and an increasing number of Internet of Things (IoT) devices. These infrastructures support critical services across enterprises, research institutions, and public organizations. As networks expand in size and complexity, administrators must continuously manage device configurations, monitor performance metrics, troubleshoot faults, and enforce security policies. Traditionally, these tasks are performed manually through command-line interfaces and specialized management tools. While such approaches provide precise control over network devices, they are often time-consuming, error-prone, and difficult to scale.

To address these challenges, the field of network automation has gained increasing attention in recent years. Automation tools and frameworks allow administrators to programmatically configure devices, collect telemetry data, and perform routine operational tasks. Technologies such as configuration management systems, scripting libraries, and workflow orchestration platforms have improved the efficiency of network operations. However, most existing automation solutions still require significant technical expertise, including knowledge of programming languages, scripting frameworks, and network protocols. Consequently, the adoption of automation technologies remains limited among many organizations that lack specialized skills.

At the same time, advances in Artificial Intelligence (AI), particularly in large language models and autonomous software agents, have proven the potential to transform complex technical workflows. AI-driven systems can interpret human language, analyze contextual information, and generate actions that assist users in solving technical problems. The emergence of agentic AI frameworks has further expanded this capability by enabling systems to reason about tasks, plan actions, and interact with external tools or platforms.

Despite these developments, the integration of AI technologies into network management remains limited compared to other areas of computing. Many current solutions rely on rule-based automation or static scripts that lack adaptive decision-making capabilities. As networks continue to grow in scale and heterogeneity, there is a need for intelligent management systems capable of assisting administrators in performing monitoring, diagnostics, and configuration tasks more efficiently.

This project explores the feasibility of combining agentic AI frameworks with workflow-based automation platforms to create a semi-autonomous network management

system. The proposed system integrates AI agents capable of interpreting user requests with a workflow orchestration layer that validates and executes network operations in a controlled environment. Through this architecture, the system aims to support natural language interaction while maintaining reliability and operational safety.

The system will be implemented and evaluated within a simulated network environment using GNS3 and virtual network devices. This environment will replicate realistic enterprise networking scenarios and enable controlled experimentation with AI-assisted network management techniques.

1.2 Statement of the Problem

Network management is still a complex and labor-intensive task that requires continuous monitoring, troubleshooting, and configuration of multiple devices across distributed infrastructures. As networks scale to support modern applications and services, manual management approaches become increasingly inefficient and susceptible to human error. Even with the availability of automation tools, many existing solutions require administrators to develop and maintain scripts or complex workflows, which can be challenging for individuals without extensive programming expertise.

Furthermore, traditional automation frameworks operate primarily through predefined instructions and lack the ability to interpret user intent or adapt dynamically to changing situations. As a result, administrators must still translate operational requirements into specific technical commands or scripts before automation tools can execute them. This limitation reduces the accessibility and flexibility of automation technologies in network operations.

While recent advancements in AI and agent-based systems demonstrate the ability to interpret natural language and assist with complex tasks, the application of these technologies in network infrastructure management remains under explored. Integrating AI-driven reasoning with structured automation workflows presents both technical challenges and opportunities for innovation. Without a controlled orchestration layer, AI-generated actions may introduce risks to network stability or security.

Therefore, a key problem addressed in this project is how to design and implement an intelligent system that combines AI-driven reasoning with workflow-based validation mechanisms to support automated network monitoring, diagnostics, and configuration tasks. Such a system must provide a balance between intelligent assistance and controlled execution to ensure safe and reliable operation within network environments.

1.3 Aim and Objectives

Aim

The aim of this project is to design and implement an agentic AI-driven automation system capable of assisting with network and IoT device management through intelligent interpretation of user requests and controlled execution of automation workflows.

Objectives

To achieve this aim, the project will pursue the following objectives:

- To design a system architecture that integrates agentic AI frameworks with a workflow automation platform for network management tasks.
- To develop a simulated network environment using GNS3 and virtual networking devices for testing and evaluating the proposed system.
- To implement AI agents capable of interpreting natural language commands related to network monitoring, diagnostics, and device configuration.
- To develop automation workflows using the n8n workflow automation platform that interact with network devices through programmatic interfaces.
- To set up a validation and control layer that ensures AI-generated actions are executed safely and systematically within the network environment.
- To evaluate the feasibility and effectiveness of the proposed system through experimental scenarios involving common network management tasks.

1.4 Significance of the Study

The increasing complexity of modern network infrastructures highlights the need for more efficient and intelligent management solutions. By exploring the integration of agentic AI frameworks with workflow-based automation platforms, this study contributes to ongoing efforts aimed at enhancing the capabilities of network automation systems.

First, the project demonstrates how AI-driven reasoning can be applied to interpret natural language instructions and translate them into structured automation tasks. This capability has the potential to reduce the technical barrier associated with traditional network automation tools, making automation more accessible to administrators who may not possess extensive programming experience.

Second, the proposed architecture introduces a controlled orchestration layer that validates and manages automation workflows before they are executed on network devices.

This design approach addresses concerns related to reliability and operational safety when incorporating AI-driven decision-making into infrastructure management.

Third, the project provides a practical prototype that illustrates how AI agents, automation workflows, and simulated network environments can be integrated to support intelligent infrastructure management. The results of this study may inform future research and development efforts in the areas of AI-assisted network operations, autonomous systems, and intelligent infrastructure management.

Finally, the findings may be valuable for both academic and practical contexts. From an academic perspective, the project contributes to the exploration of emerging technologies at the intersection of artificial intelligence and network engineering. From a practical perspective, the system demonstrates how intelligent automation techniques can assist network administrators in managing increasingly complex infrastructures.

1.5 Scope and Limitations

This project focuses on the design and implementation of a prototype system that integrates agentic AI frameworks with a workflow automation platform for network management tasks. The system will be evaluated within a simulated environment that replicates typical enterprise networking scenarios using GNS3 and virtual network devices.

The scope of the project includes the development of a simulated network topology, the implementation of AI agents capable of interpreting user commands, and the creation of automation workflows that interact with network devices for monitoring, diagnostics, and configuration purposes. The system will provide a chatbot-based interface through which users can submit natural language requests that are processed by the AI agent and translated into appropriate automation actions.

However, several limitations are acknowledged. First, the system will operate within a simulated network environment rather than a production network infrastructure. Consequently, performance results and operational behavior may differ from real-world deployments. Second, the project focuses on demonstrating the feasibility of the proposed architecture rather than developing a fully autonomous network management system. Third, the capabilities of the AI agent depend on the underlying language model and predefined workflows, which may limit the system's ability to handle extraordinarily complex or unforeseen scenarios.

Despite these limitations, the project aims to provide a meaningful demonstration of how agentic AI systems can be integrated with automation frameworks to support intelligent infrastructure management.

1.6 Structure of the Report

The remainder of this report is organized as follows. Chapter 2 presents a review of related literature and existing research in the fields of network automation, artificial intelligence in network management, and agent-based systems. This chapter identifies key technological developments and highlights gaps that motivate the proposed approach.

Chapter 3 describes the overall system architecture and methodology of the proposed solution, including the integration of AI agents, workflow automation platforms, and the simulated network environment.

Chapter 4 details the implementation of the system, including the development of automation workflows, integration of AI components, and configuration of the experimental network environment.

Chapter 5 presents the experimental evaluation and results obtained from testing the system in various network management scenarios.

Chapter 6 discusses the findings of the study, including system performance, limitations, and lessons learned during the implementation process.

Finally, Chapter 7 concludes the report by summarizing the key contributions of the project and outlining potential directions for future research and development.

Chapter 2

Literature Review

2.1 Traditional Network Management Approaches

Traditional network management has historically relied on command-line configuration, device-specific administrative interfaces, and polling-based monitoring systems such as SNMP. These methods provide direct control over infrastructure but require substantial technical expertise and continuous manual intervention. In complex environments, this approach becomes difficult to scale because each device may require individual attention, and operational visibility is often fragmented across tools and interfaces.

The broader networking literature has long recognized these limitations. Kreutz et al. argue that conventional IP networks are difficult to configure and reconfigure because control logic remains tightly coupled to individual devices and policies are expressed through low-level commands [9]. This operational model constrains flexibility and slows response to changing network conditions.

In practical terms, traditional network management remains strong where deterministic control is required, but it performs poorly when operators must act quickly across many devices or respond to complex and evolving conditions. These weaknesses motivate the transition toward programmable automation and higher-level orchestration.

2.2 Network Automation and Programmability

Network automation represents a major improvement over purely manual operations. Fuzi et al. show that Ansible-based scripting can automate EIGRP deployment, addressing, static routing, and verification across multiple routers in a simulated environment [6]. Their study demonstrates the benefits of infrastructure-as-code approaches, including consistency, repeatability, reduced configuration error, and lower administrative effort.

The significance of this literature lies in its demonstration that network tasks can be represented declaratively and executed through reusable automation artifacts. The same principle extends to broader programmability paradigms, including SDN and model-driven management. Programmability allows administrators to work with abstractions, templates, and centralized logic rather than repeating device-by-device command sequences.

However, automation by itself does not eliminate the need for interpretation and decision-making. Script-based approaches work best when tasks and conditions are known in advance. They are less effective when operators must interpret ambiguous requests, diagnose unfamiliar problems, or select among alternative responses. This limitation creates a natural opening for AI-assisted layers above deterministic automation.

2.3 Intent-Based Networking

Intent-based networking (IBN) attempts to raise the abstraction level of network management by allowing operators to specify desired outcomes rather than low-level commands. In principle, an IBN system translates business or operational intent into network policies and configuration states. This idea is important because it shifts the focus from syntax to objectives and creates a conceptual bridge between human requirements and machine execution.

Although the supplied folder included a document on intent-based networking, extractable metadata from that file was incomplete and should be verified before formal citation. Even so, the concept remains relevant because later LLM-based network management research can be understood as extending the IBN vision. Where IBN focuses on mapping intent into policy, LLM-based systems add natural-language interpretation, reasoning over unstructured artifacts, and more flexible task decomposition.

For the present project, the significance of IBN is primarily conceptual. It supports the view that administrators should not always have to encode their goals in low-level automation scripts. Instead, systems should be designed to infer and operationalize those goals within controlled boundaries.

2.4 Artificial Intelligence in Network Management

Artificial intelligence in network management predates LLMs and includes applications of machine learning, deep learning, reinforcement learning, anomaly detection, and predictive analytics. This body of work has focused on traffic analysis, performance optimization, fault detection, and decision support. It has helped establish that adaptive methods can outperform static rules in environments where patterns change over time.

Some of the supplied sources, including a recent exploratory preprint and shorter overview articles, reflect this broader trend. Although not all of them are equally rigorous, they collectively indicate that AI techniques are increasingly used to support predictive maintenance, traffic engineering, anomaly detection, and optimization in communication networks [10, 4, 8].

The limitation of much pre-LLM AI work is that it often addresses narrow tasks and depends on structured data, carefully defined training pipelines, or specific optimization targets. It does not necessarily provide the flexible, conversational, and multi-step reasoning capabilities needed for broader operational assistance. This helps explain why LLM-based approaches have become an important new area of attention.

2.5 Large Language Models in Network Operations

Large language models introduce a qualitatively different capability into network operations. Hong et al. show that LLMs can interpret natural-language requests, work with unstructured sources such as documentation and logs, and support broader management and operations workflows [7]. Their survey positions LLMs as a promising interface layer between human operators and increasingly complex infrastructures.

This is directly relevant to the proposed project. An LLM can interpret requests such as checking routing health, diagnosing misconfiguration, or generating an initial automation plan from natural language. It can also summarize technical findings for users in accessible language. In this sense, LLMs can lower the expertise threshold associated with traditional automation.

At the same time, the literature is explicit about the risks. Hallucination, unreliable command generation, domain adaptation challenges, and safety concerns are major constraints on real-world deployment. LLMs therefore require grounding and bounded execution. The most responsible interpretation of the literature is that LLMs should augment deterministic tools rather than replace them.

The short paper by Abane, Battou, and Merzouki further strengthens this point by proposing a network management assistant grounded in prompt engineering, retrieval-augmented generation, and knowledge graphs [1]. Its key contribution is not just the use of an LLM, but the recognition that practical network assistance must be adapted at runtime to actual network state, documentation, and platform-specific knowledge.

2.6 Agentic Systems and Workflow Orchestration

Agentic systems extend beyond simple question answering by decomposing tasks, selecting actions, using tools, and coordinating execution across multiple steps. In the context of network management, this means an AI component may interpret user intent, trigger diagnostics, analyze output, formulate a remediation plan, and hand off approved steps to an automation framework.

This architectural logic aligns closely with the project under study. The project outline proposes specialized functions such as configuration generation, diagnostics, remediation, and reporting, all mediated through workflow orchestration. From a systems perspective, workflow tools provide the sequencing, logging, validation, and repeatability needed to convert high-level reasoning into auditable execution.

The literature supplied for orchestration platforms is weaker than the networking literature, and some n8n-related sources appear non-academic. Accordingly, the role of workflow orchestration should be justified primarily as an engineering necessity rather than as a deeply theorized research domain. Nevertheless, the integration of reasoning and deterministic workflow execution is one of the most practically important design insights

supported by the overall source set.

2.7 AI-Driven IoT Network Management

IoT network management introduces additional complexity because devices are heterogeneous, resource-constrained, and often deployed under unstable or lossy conditions. Aboubakar et al. provide a strong survey of these issues, showing that efficient IoT management must address scalability, fault tolerance, quality of service, energy efficiency, security, and self-configuration [2].

This literature is highly relevant because the project does not focus solely on routers and switches. A useful network and IoT management platform must operate across different protocols, capabilities, and management surfaces. Static playbooks alone are unlikely to accommodate this diversity effectively.

Du et al., through *LLMind 2.0*, extend AI-based management into a distributed IoT automation setting [5]. Their proposed use of lightweight local LLM agents and natural-language machine-to-machine communication is particularly important because it addresses problems of scalability, device heterogeneity, and privacy. Although the paper is currently a preprint and therefore should be treated with appropriate caution, it provides a forward-looking model for how AI-assisted IoT orchestration may evolve beyond centralized control.

Together, these sources suggest that any future-ready implementation should account not only for centralized orchestration but also for the possibility of distributed intelligence near the device layer.

2.8 Research Gaps

The literature reveals several gaps that justify the proposed project. First, there remains a practical integration gap between deterministic network automation tools and LLM-based reasoning systems. While many papers discuss the potential of AI for network operations, fewer demonstrate controlled end-to-end architectures that combine natural-language interaction, workflow validation, and safe infrastructure execution.

Second, there is a gap between observability and action. Modern telemetry and model-driven monitoring provide rich operational data, but transforming that data into trustworthy diagnostic and remediation workflows remains challenging. This is especially true when AI components are introduced into the decision loop [3].

Third, enterprise network automation and IoT device management are often treated as separate domains. The proposed project is significant because it attempts to integrate both within a unified architecture. Finally, there is limited validation in realistic network testbeds of hybrid systems that combine AI interpretation with constrained execution. This gap is particularly important for demonstrating feasibility, safety, and operational

usefulness.

Overall, the literature supports the need for a hybrid architecture in which deterministic automation handles execution, telemetry and documentation provide grounding, and AI agents support intent interpretation, diagnosis, planning, and user-facing explanation.

Review Matrix

Table 2.1: Review matrix of the principal sources used in the literature review.

Paper	**Topic**	**Method / Type**	**Main Findings**	**Limitations**	**Relevance**
Fuzi et al. (2021)	Network automation	Implementation study using Ansible in GNS3	Shows YAML and Ansible can automate EIGRP deployment and verification with reduced manual effort	Small-scale lab; static scripts; no adaptive reasoning	Core baseline for deterministic execution
Aboubakar et al. (2022)	IoT management	Peer-reviewed survey	Identifies scalability, fault tolerance, QoS, energy efficiency, security, and self-configuration requirements	Survey only; predates recent LLM advances	Essential for IoT requirements framing
Hong et al. (2025)	LLM-based network operations	Comprehensive survey	Highlights intent interpretation, automation, use of unstructured artifacts, and hallucination risks	Survey; limited primary experiments	Strong framing source for LLM-based network management

Paper	Topic	Method / Type	Main Findings	Limitations	Relevance
Abane et al. (2024)	AI network assistant	Demo/system paper using RAG and knowledge-graph grounding	Shows runtime adaptation to platform and network state is feasible	Early-stage demonstrator; limited scale evaluation	Strong precedent for grounded AI assistance
Du et al. (2025)	Distributed IoT automation	arXiv research paper	Argues distributed device agents improve scalability, privacy, and heterogeneity handling	Preprint; requires further validation	Highly relevant for multi-agent IoT extensions
Kreutz et al. (2014/2015)	SDN	Foundational survey	Establishes programmability and separation of control and data planes as key to easier management	Not AI-focused	Important theoretical background for programmable control
Cisco (2024)	Telemetry	Vendor white paper	Explains streaming telemetry, YANG, gNMI/gRPC, and observability benefits	Not peer reviewed; vendor-authored	Useful technical support for the monitoring and diagnostics layer
Okeda (2025)	AI-driven network management overview	Preprint literature review	Broad overview of AI in network management and optimization	Not peer reviewed; secondary source	Background only; not a core authority

Paper	Topic	Method / Type	Main Findings	Limitations	Relevance
Das (2024)	AI for IoT management	Short article/review	Summarizes AI techniques for IoT management	Limited methodological rigor	Peripheral support only
Santhosh and Gary (2023)	Self-healing networks	Conceptual article	Discusses anomaly detection and automated remediation goals	Weak scholarly strength	Use only as weak background

Chapter 3

System Architecture and Design

This chapter presents the architecture and design of the proposed AI-assisted automation system. It explains how the main components of the project are organized, how user requests move through the system, and how AI reasoning is separated from infrastructure execution. The chapter also describes the virtualized environment, the layered interaction model, the role of the AI agent, and the n8n workflow orchestration layer.

3.1 Overall System Architecture

The proposed system implements an AI-assisted automation platform for managing network, systems, and IoT infrastructure within a simulated enterprise environment. The architecture integrates artificial intelligence reasoning, workflow orchestration, programmable network automation tools, and virtualized systems administration workflows to enable users to perform monitoring, diagnostics, configuration, and administrative tasks using natural language commands.

This architectural approach was selected to separate natural-language reasoning from infrastructure execution, thereby improving controllability, auditability, and operational safety in a semi-autonomous network management environment. By decoupling decision-making from execution, the system ensures that AI-generated actions are validated before being applied to network devices.

The architecture is organized into two main simulation layers. The first is the **Network Layer**, which is implemented through GNS3 and represents the enterprise network simulation environment. This layer supports router, switch, firewall, VLAN, ACL, and network-policy validation workflows. The second is the **System Administration Layer**, which is implemented through Proxmox and represents the company system simulation environment. This layer supports virtual machine administration, company endpoint monitoring, user-policy enforcement, and AI-assisted systems administration workflows.

3.1.1 Virtualized Infrastructure Environment

To support experimentation and realistic infrastructure behavior, the system is deployed within a virtualized lab environment hosted on a Proxmox VE hypervisor. Virtualization is employed to enable reproducibility of experiments, isolation of system components, efficient hardware utilization, and safe testing of automation workflows without impacting production infrastructure.

Proxmox was selected over simpler single-host configurations due to its ability to manage multiple virtual machines with isolated resources, support complex networking setups, and provide a scalable environment suitable for simulating enterprise-level systems. The Proxmox host acts as the central infrastructure layer responsible for running all components of the project environment.

Proxmox is also used to represent the System Administration Layer of the project. This means it is not only a virtualization platform, but also the simulated company systems environment used for systems administration experiments. Through Proxmox, the project can test virtual machine administration, company endpoint monitoring, user activity tracking, policy enforcement, and AI-assisted systems administration workflows in a controlled lab environment.

3.1.2 Virtual Machine Architecture

Within the Proxmox host, six primary virtual machines are deployed to support the system architecture:

1. **Ubuntu Server VM (Automation Stack):** This virtual machine hosts the core automation services used in the project. It runs Docker containers that deploy key services including the n8n workflow automation engine, Portainer container management interface, and Cloudflare Tunnel service. This VM functions as the automation and orchestration hub, where AI-generated actions are translated into structured workflows and executed in a controlled manner.

2. **Ollama VM (AI Model Serving Layer):** The Ollama virtual machine provides the model serving layer for the system. It hosts locally deployed large language models (LLMs) used by the AI agent for intent interpretation, decision-making, and response generation. This separation of the model serving layer from the automation and orchestration components improves modularity, scalability, and performance by isolating computationally intensive AI processing from workflow execution.

3. **Ubuntu Test VM (Company System Simulation Endpoint):** The Ubuntu Test VM represents a company-managed endpoint used for systems administration and security monitoring experiments. It is used to simulate company users, restricted user groups, administrator accounts, and company policy enforcement through automated scripts and AI-assisted workflows. In this role, the Ubuntu Test VM acts as an Active Directory-like simulation environment where workplace user activity, restricted actions, and policy violations can be monitored and evaluated. This VM is an important part of the System Administration Layer.

4. **GNS3 VM (Network Simulation Backend):** The GNS3 virtual machine provides the network simulation environment used to emulate routers, switches, firewalls, and other infrastructure devices. GNS3 was selected because it enables

realistic simulation of enterprise networking environments using virtual appliances, allowing the system to operate in conditions that closely resemble real-world network deployments.

5. **Ubuntu Desktop VM (Shared Lab Workstation):** The Ubuntu Desktop virtual machine acts as a shared graphical workstation for the project team. It hosts the GNS3 client application and network management tools, enabling users to interact visually with the simulated network environment through remote access mechanisms such as RDP.

6. **IoT Core VM:** The IoT Core VM is added as the IoT station inside the Proxmox environment. It represents the IoT experimentation layer of the project and provides a dedicated machine for IoT device simulation, monitoring, and future integration with automation workflows. This VM supports the broader project goal of combining network and IoT device management within the same AI-assisted automation architecture.

Together, these virtual machines show that the architecture is not limited to network simulation only. The Proxmox environment supports both the System Administration Layer, represented by the Ubuntu Test VM and systems administration workflows, and the Network Layer, represented by the GNS3 VM and simulated network devices. The IoT Core VM extends the architecture by adding a dedicated environment for IoT-related experimentation.

The overall virtualized system architecture is illustrated in Figure 3.1.

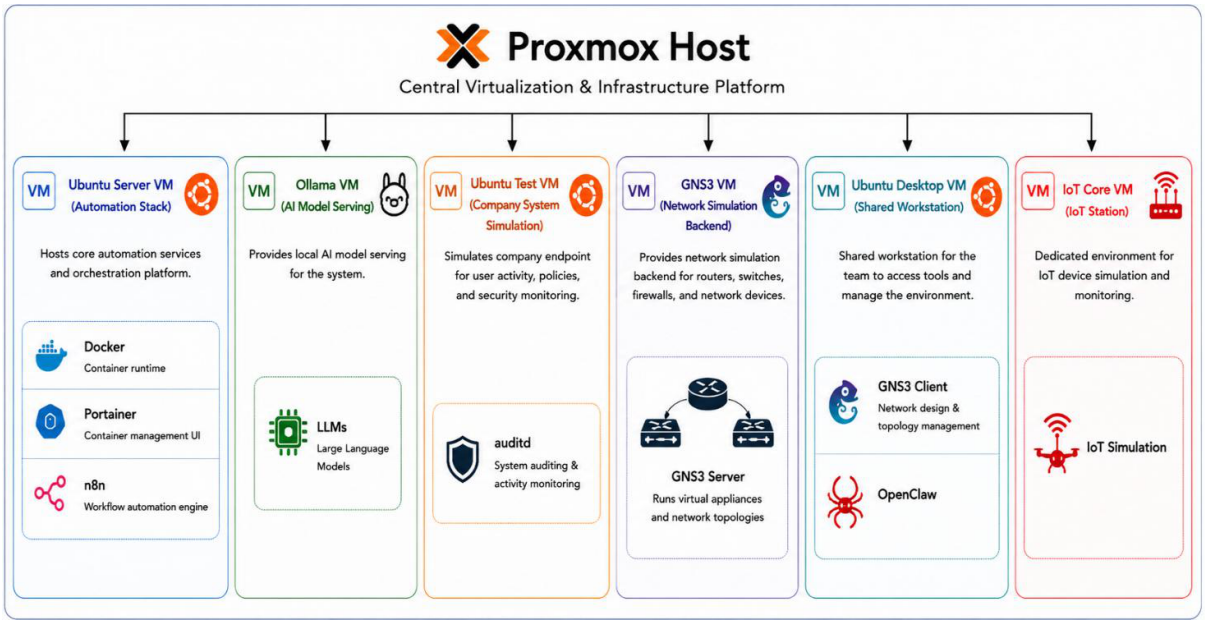


Figure 3.1: Virtualized system architecture showing the Proxmox host and deployed virtual machines.

3.1.3 Layered Interaction Model

In addition to the physical infrastructure layout, the system follows a layered interaction model that defines how user requests are processed and executed across different components. The model separates user interaction, AI reasoning, workflow orchestration, automation execution, and the simulated infrastructure environment.

The project has two main interaction paths. The first path represents the System Administration Layer, where Proxmox is used as the company system simulation environment. The second path represents the Network Layer, where GNS3 is used as the network simulation environment.

For the System Administration Layer, the process follows this sequence:

Administrator → System Admin Chat Interface → AI Agent → n8n Workflow Engine → Automation Scripts → Proxmox Hosts

This interaction process is illustrated in Figure 3.2.

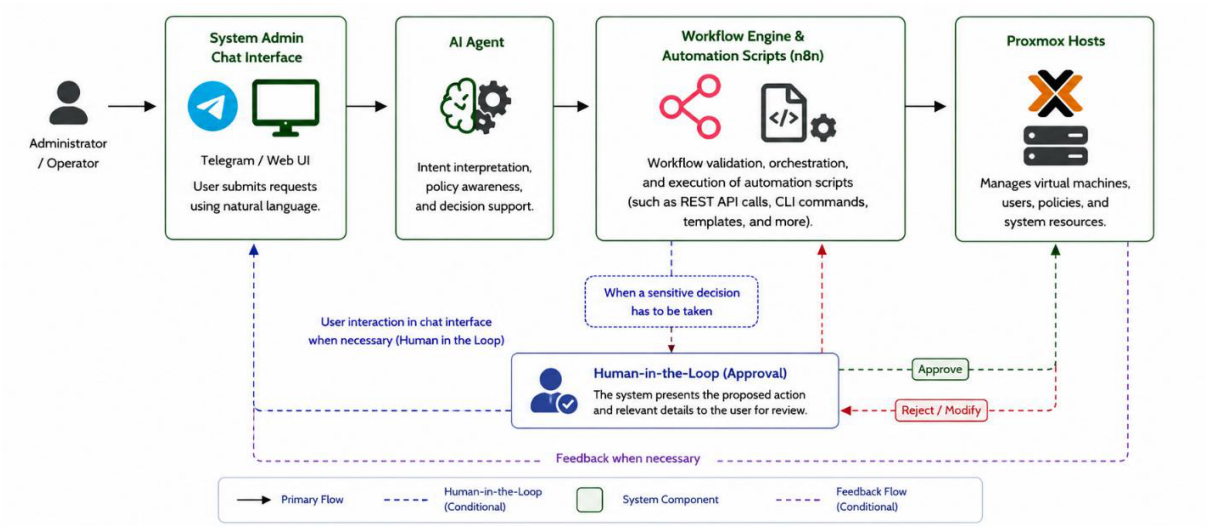


Figure 3.2: System administration layer interaction model showing the flow from user request to Proxmox-based company system simulation.

The system administration interaction model shows how an administrator or operator submits requests through the chat interface, such as Telegram or a web UI. The AI agent interprets the request, identifies the intended administrative task, and passes the structured request to the n8n workflow engine. The n8n layer validates and orchestrates the workflow before automation scripts execute the required system administration actions. These actions interact with the Proxmox environment, where virtual machines, simulated company users, endpoint monitoring, policy enforcement, and audit logs are managed.

For the Network Layer, the conceptual flow is:

User → Chat Interface → AI Agent → n8n Workflow Engine → Python Automation Layer → GNS3 Environment → Network Devices

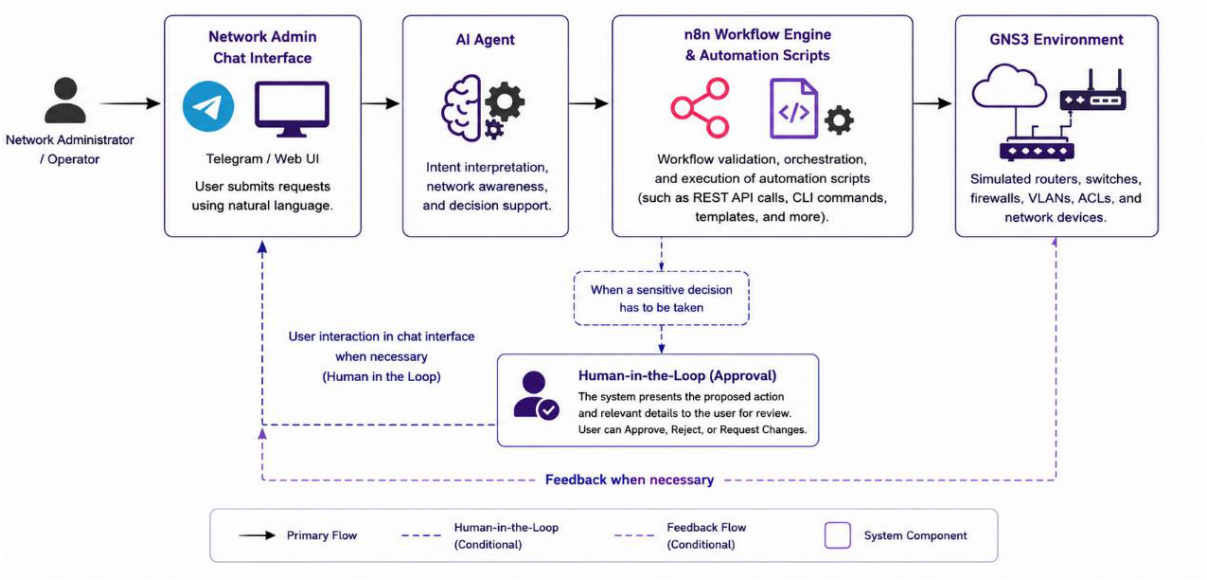


Figure 3.3: Network layer interaction model showing the layered process from user input to GNS3-based network device management.

The network interaction model shows how network-related requests are processed. A user submits a natural-language request through the chat interface. The AI agent interprets the request and identifies the required network operation. The n8n workflow engine validates and routes the request to the appropriate workflow. The Python automation layer then interacts with the GNS3 environment, where simulated routers, switches, firewalls, VLANs, ACLs, and network devices are managed.

3.1.4 System Operation Flow

The process begins when a user submits a natural language request through a chatbot interface. The request is forwarded to the AI agent, which interprets user intent using a large language model.

The AI agent then determines the appropriate action and triggers a corresponding workflow in the n8n automation engine. The orchestration layer executes structured workflows that invoke Python-based automation scripts, which interact with simulated network devices through automation libraries such as Netmiko, Paramiko, and NAPALM. The results of the operation are then returned through the same processing chain, allowing the AI agent to generate a human-readable response for the user.

This layered interaction model ensures a clear separation between:

- user interaction,
- intelligent decision-making,
- workflow execution, and
- device-level control.

3.1.5 Operational Safety and Human Oversight

To ensure operational safety, the system incorporates a Human-in-the-Loop (HITL) validation mechanism within the automation pipeline.

For sensitive operations, such as configuration changes or infrastructure modifications, the system requires manual approval before execution. This ensures that critical actions are verified by a human operator before being applied to the network environment, reducing the risk of unintended or harmful changes.

3.1.6 Architectural Summary

Together, these infrastructure components form a layered architecture in which AI reasoning, automation workflows, and network simulation operate as interconnected but decoupled subsystems.

This design enables the system to combine the flexibility of AI-driven decision-making with the reliability and safety of structured automation, making it suitable for semi-autonomous network and IoT management.

3.2 Design Principles

The system architecture was developed according to several key design principles that support reliability, extensibility, and operational safety. These principles are directly reflected in the structure and operation of the proposed system rather than being applied in an abstract manner.

Modularity

Modularity is achieved by separating the system into distinct functional layers, including the AI reasoning layer, the n8n workflow orchestration layer, and the network automation layer.

This separation allows each component to operate independently while maintaining coordinated system functionality. For instance, the AI agent responsible for interpreting user intent operates independently of the workflow execution layer, meaning that different language models can be integrated or replaced without modifying the n8n workflows. Similarly, updates to automation scripts or network libraries do not require changes to the AI reasoning component. This modular structure improves maintainability, simplifies debugging, and enables incremental development of the system.

Scalability

Scalability is supported through the use of virtualization and containerization technologies within the system architecture.

The Proxmox hypervisor enables the deployment of additional virtual machines as system requirements grow, while Docker allows services such as n8n to be replicated, extended, or redeployed with minimal configuration overhead. This design allows the system to scale from a small experimental lab environment to a more complex enterprise-like setup, supporting multiple network segments, services, and concurrent users.

Extensibility

Extensibility is achieved through the flexible integration of new components into the system architecture.

The workflow-based design of n8n allows new automation processes to be introduced without disrupting existing workflows. Additionally, the AI layer can integrate different language models through the Ollama model-serving environment, enabling the system to adapt to evolving AI capabilities. This ensures that the architecture can accommodate future enhancements, including new automation tools, additional network devices, and expanded IoT use cases.

Security Model

The system adopts a Zero Trust security model to protect internal infrastructure while enabling remote access.

To reduce external attack exposure, the architecture uses outbound tunnel-based connectivity instead of directly exposing internal services through port forwarding or public IP addresses. This ensures that core services such as the Proxmox management interface, n8n automation engine, and GNS3 environment remain inaccessible to unsolicited external traffic. By eliminating direct inbound connections, the system significantly reduces its attack surface and improves overall security.

Access Control

Access control is implemented through identity-based authentication using Cloudflare Access. Rather than relying solely on network-level access, users must authenticate their identity before accessing any system component. This ensures that only authorized team members can interact with the infrastructure.

Authentication occurs before reaching the application layer, providing an additional level of protection regardless of whether service endpoints are publicly accessible.

Protected Communication

All communication between users and system components is secured using encrypted channels. Cloudflare Tunnel provides TLS-based encryption for all traffic, ensuring that sensitive data such as authentication credentials, automation commands, and system

responses cannot be intercepted or modified during transmission. This approach guarantees the confidentiality and integrity of communications while supporting secure remote collaboration within the system.

3.3 Agentic Workflow Design

A key feature of the proposed system is the implementation of agentic workflows, which combine AI-driven reasoning with structured workflow-based execution.

In this study, an agentic workflow is defined as a bounded automation process in which an AI component interprets user intent, selects an approved workflow path, and operates within predefined execution constraints enforced by the orchestration layer.

This approach enables the system to translate natural language inputs into controlled automation tasks while maintaining operational safety and predictability.

3.3.1 Role of the AI Agent

The AI agent serves as the intelligent interface between the user and the automation system. Unlike traditional chatbots that generate only conversational responses, the agent in this system performs intent interpretation and decision support, enabling it to initiate operational workflows.

The agent does not directly execute commands on network devices. Instead, it translates user requests into structured workflow triggers that are processed by the orchestration layer, ensuring that all actions are validated before execution.

3.3.2 Agent Reasoning Model (Observe–Think–Decide–Act)

The behavior of the AI agent is implemented using a structured reasoning model based on the Observe–Think–Decide–Act (OTDA) cycle. Within the system architecture, this cycle functions as a formal sub-process governing how user requests are interpreted and executed:

1. **Observe** – The agent receives and parses the user's natural language request.
2. **Think** – The agent analyzes the request using a large language model to determine intent and relevant context.
3. **Decide** – The agent selects the appropriate predefined workflow required to satisfy the request.
4. **Act** – The agent triggers the selected workflow through the n8n orchestration layer.

This structured reasoning model ensures consistency in decision-making and improves traceability of system actions. The internal reasoning mechanism of the AI agent follows this structured Observe–Think–Decide–Act cycle, as illustrated in Figure 3.4.

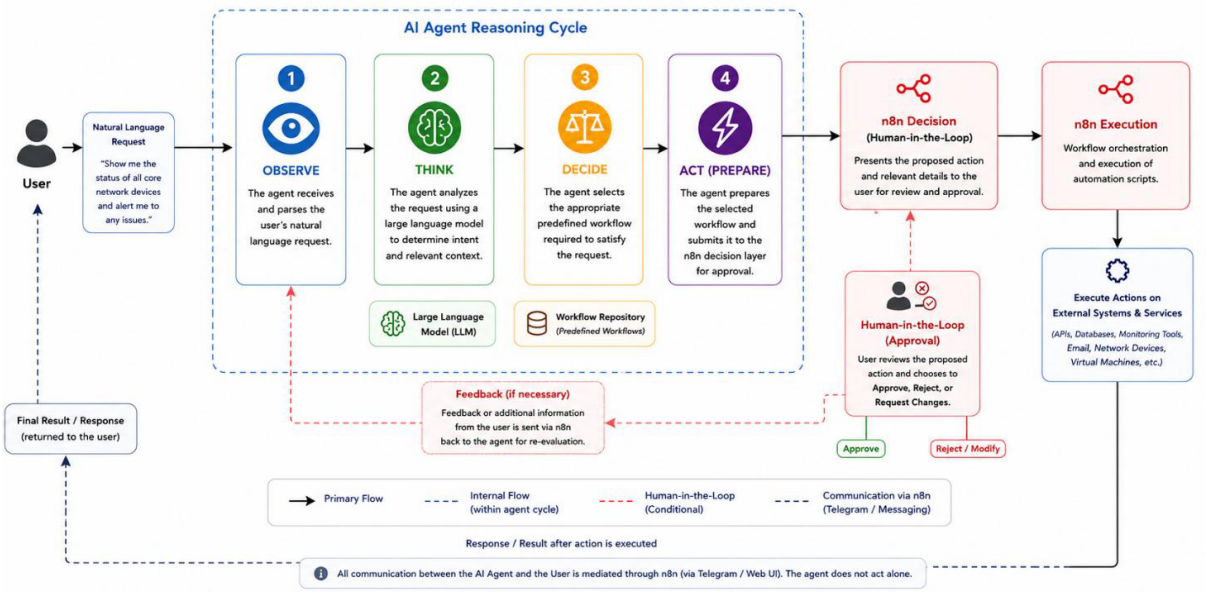


Figure 3.4: AI agent reasoning cycle showing the Observe–Think–Decide–Act process, human-in-the-loop approval, n8n decision handling, and execution feedback.

Intent Interpretation

Intent interpretation is performed by the AI model, which extracts actionable meaning from user input and maps it to a predefined workflow.

For example, when a user submits a request such as "Check the status of Router 1 interfaces", the agent identifies the target device, the required operation, and the corresponding diagnostic workflow. The interpreted intent is then converted into a structured request that can be processed by the automation layer.

Task Planning and Decomposition

Once the intent is identified, the agent decomposes the request into a sequence of executable steps aligned with predefined workflows. For instance, a diagnostic request may involve identifying the target device, establishing a connection to the device, executing diagnostic commands, collecting output data, and forwarding results for interpretation.

This decomposition ensures that complex tasks are executed in a structured, repeatable, and verifiable manner.

Controlled Execution via Orchestration Layer

A fundamental design decision in the system is that the AI agent does not directly interact with network devices. Instead, all actions are routed through the n8n orchestration layer, which functions as a control and validation mechanism.

This design serves as the primary safeguard against unsafe or uncontrolled AI behavior. It ensures that only predefined workflows can be executed, arbitrary or unverified commands are prevented, and all actions follow a structured and auditable execution

process.

This approach directly addresses known limitations of large language models, including hallucinated or incorrect command generation, lack of deterministic execution behavior, and risks associated with unconstrained autonomous operation.

By enforcing workflow-based execution, the system maintains operational safety while still leveraging AI capabilities.

Safety Mechanisms and Human-in-the-Loop

To further enhance operational safety, the system incorporates guardrails and a Human-in-the-Loop (HITL) validation mechanism.

The system distinguishes between different levels of operational risk:

- Low-risk operations (e.g., monitoring and diagnostics) are executed automatically.
- High-risk operations (e.g., configuration changes) require manual approval before execution.

For example, a request such as "Check routing table on Router 1" triggers a read-only workflow that executes without user intervention. By contrast, requests such as "Update VLAN configuration on Switch 1" or "Modify routing parameters" involve altering network state, so the workflow pauses before execution, a human operator reviews the requested action, and execution proceeds only after approval.

This mechanism ensures that potentially disruptive changes are verified before being applied, reducing the risk of unintended network instability.

Feedback Loop and Result Interpretation

After workflow execution, the system returns the resulting data to the AI agent, forming a closed feedback loop. The agent analyzes the technical output, verifies whether the requested task was successful, and translates the output into a human-readable response.

For example, instead of returning raw command output, the system may generate a response such as "Router 1 interfaces are operational. No connectivity issues detected." This feedback loop improves usability while maintaining technical accuracy and supports continuous interaction between the user and the system.

3.4 Automation Orchestration Layer

The n8n workflow automation platform serves as the central orchestration layer of the system and functions as the automation backbone. It is responsible for executing infrastructure tasks triggered by the AI agent in a structured, controlled, and secure manner.

Within the system architecture, n8n operates as an intermediary layer between the agentic AI framework and the network infrastructure, ensuring that high-level reasoning is translated into validated and deterministic technical actions.

3.4.1 The Orchestration Bridge

The n8n platform serves as the workflow orchestration layer responsible for sequencing, validating, and executing approved automation tasks within the system architecture.

It operates as an intermediary between the AI reasoning layer and the network automation layer, ensuring that high-level decisions generated by the AI agent are translated into structured, controlled, and verifiable execution processes.

3.4.2 Separation of Reasoning and Execution

A key architectural design principle of the system is the separation between AI-driven decision-making and workflow-based execution.

When a user submits a request, the AI agent interprets the intent and generates a structured instruction. This instruction is then passed to the n8n orchestration layer, which triggers a predefined workflow corresponding to the requested operation.

This separation ensures that the AI agent performs interpretation and decision support, while the orchestration layer enforces execution constraints and validation. By isolating execution from reasoning, the system prevents direct interaction between AI-generated outputs and network devices, thereby improving reliability and operational safety.

3.4.3 Workflow Execution Model

Automation in n8n is implemented through structured workflows composed of interconnected nodes that execute tasks in a defined sequence.

Workflows are typically triggered through API requests or webhooks generated by the AI agent. Each workflow consists of multiple nodes representing operations such as script execution, API calls, data processing, and interaction with infrastructure services.

n8n manages the flow of data between nodes, enabling outputs from one step to be used as inputs for subsequent steps. Conditional logic mechanisms, such as IF and Switch nodes, allow workflows to dynamically adapt based on real-time conditions. For example, a workflow may verify device reachability before proceeding with diagnostic operations.

The collected data is first processed by the n8n orchestration engine before being returned to the AI agent. At this stage, n8n performs several preprocessing tasks, including logging, error detection, and data formatting into structured forms such as JSON. This transformation improves consistency and enables reliable downstream analysis.

AI Interpretation and Analysis (Interpreted Output)

Once structured, the data is passed to the AI agent, corresponding to the "Observe" phase of the reasoning cycle. The AI agent analyzes the output to verify whether the requested task was successful, detect anomalies or failures, and perform basic root cause analysis.

For example, if an interface is reported as down, the AI may infer potential causes such as configuration errors or connectivity issues. The final output is then translated into a human-readable explanation.

Feedback and Verification Loop

To enhance system reliability, the architecture incorporates a feedback and verification loop. After execution, results are evaluated against expected outcomes, inconsistencies may trigger additional diagnostics or alerts, and the AI agent may initiate further workflows based on observed conditions.

At the current stage of the project, this feedback mechanism is partially implemented. Result evaluation and response generation are operational, while automated re-triggering of corrective workflows is under development.

This approach moves the system toward a closed-loop automation model, where actions can be refined iteratively based on real-time network state.

Final Communication with the User

Once analysis is complete, the AI agent generates a natural language summary of the results and delivers it to the user.

The response typically includes the actions performed, current network status, detected issues, and recommended next steps where applicable. By translating complex technical outputs into human-readable insights, the system improves usability while maintaining technical accuracy.

Chapter Summary

This chapter presented the system architecture and design of the proposed AI-assisted automation platform. The architecture is built on a Proxmox-based virtualized environment that supports two main simulation layers: the System Administration Layer and the Network Layer. The System Administration Layer uses Proxmox and simulated company endpoints to support virtual machine administration, user-policy enforcement, security monitoring, and systems administration workflows. The Network Layer uses GNS3 to simulate routers, switches, firewalls, VLANs, ACLs, and network management operations.

The chapter also explained the role of the AI agent, which interprets natural-language user requests and selects the appropriate workflow path. However, the AI agent does not directly execute infrastructure actions. Instead, execution is controlled through the n8n workflow orchestration layer and automation scripts, improving safety, traceability, and reliability.

Chapter 4

System Implementation

4.1 Development Environment

This section describes the technical environment used to build, deploy, and test the AI-driven network and IoT management prototype. The environment was designed to be reproducible, modular, and representative of realistic enterprise networking conditions, allowing each subsystem to be developed and evaluated in isolation while maintaining coordinated system-level integration.

4.1.1 Virtualization Platform

The development environment is built upon Proxmox VE, an open-source hypervisor platform that hosts all virtual machines required for the system. This virtualization approach was selected to support repeatable testing, component isolation, safe experimentation, and controlled interaction between the simulated network environment and the automation layers. By separating each functional subsystem into its own virtual machine, the prototype avoids cross-component interference and allows each service to be configured, restarted, or replaced independently without disrupting the rest of the environment.

Within the Proxmox host, four primary virtual machines are deployed, each serving a distinct role in the system architecture. Together, these components supported the simulation, orchestration, execution, and AI-assisted interaction functions required by the prototype:

- **Ubuntu Server VM:** Hosts the core automation services, including Docker containers running the n8n workflow orchestration engine, Portainer container management interface, and supporting backend services. This VM functions as the automation and orchestration hub.
- **Ollama VM:** Provides the AI model serving layer, hosting locally deployed large language models (LLMs) used by the AI agent for intent interpretation, decision-making, and response generation. Separating the model inference layer from the orchestration components improves modularity and avoids compute resource contention.
- **GNS3 VM:** Provides the network simulation backend, enabling realistic emulation of enterprise routing, switching, and VLAN environments using virtual Cisco IOS device images.

- **Ubuntu Desktop VM:** Acts as a shared graphical workstation hosting the GNS3 client application and network management utilities, allowing team members to interact visually with the simulated topology.

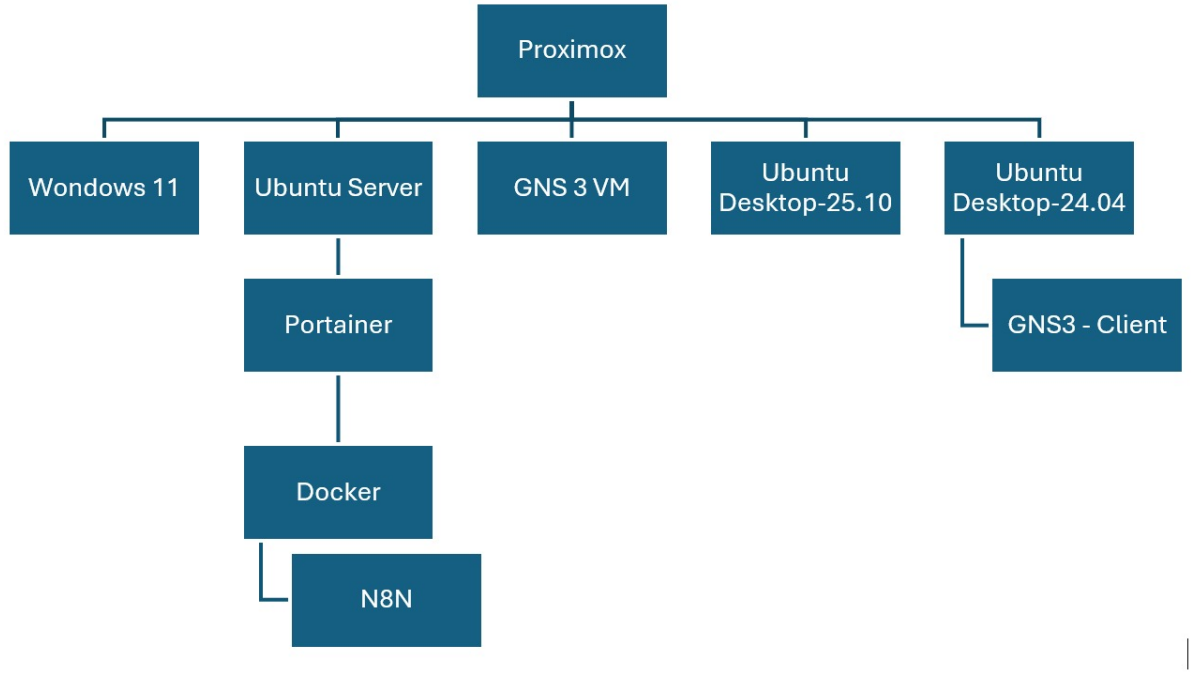


Figure 4.1: Implementation architecture showing the virtualized Proxmox host, the four deployed virtual machines, and the subcomponents hosted within each VM (n8n, Docker, Portainer, LLMs, and GNS3 Client).

The Ubuntu Server VM runs Ubuntu Server 22.04 LTS as its host operating system, selected for its long-term stability, broad package compatibility, and native support for Docker, Python automation libraries, and network management tooling. Ubuntu provides broad compatibility with the Python-based automation utilities and supporting runtime components used in the prototype. If configuration-management extensions such as Ansible are incorporated, the platform also supports them without additional configuration overhead.

4.1.2 Simulation Tools

Network and IoT simulation is implemented through a combination of purpose-built emulation platforms. GNS3 serves as the primary network simulation environment, enabling the deployment of Cisco IOS router and switch images within a graphical topology. GNS3 was selected because it supports integration with real device images and provides a realistic command-line interface accessible via SSH, making it directly suitable for testing automation scripts that interact with live network operating system instances.

For structured routing and switching scenarios, Cisco Packet Tracer is also referenced due to its built-in IoT simulation module, which supports sensors, actuators, micro-

controllers, and programmable endpoints communicating through MQTT and HTTPS. IoT device simulation is additionally supported by Cooja, the network simulator embedded within the Contiki operating system, which provides emulation of resource-constrained sensor nodes communicating over low-power wireless protocols. These tools collectively allow the testbed to represent both conventional network infrastructure and heterogeneous IoT endpoints within a controlled and repeatable environment.

4.1.3 Operating Systems

The primary host operating system for all server-side components is Ubuntu Server 22.04 LTS, selected for its stability, broad package availability, and compatibility with Docker, Python automation libraries, and network management tools. The GNS3 server is installed on a dedicated Ubuntu VM to isolate simulation workloads from the application stack, ensuring that heavy network emulation tasks do not degrade the performance of the orchestration and AI services.

Network device images within GNS3 run Cisco IOS, providing realistic routing, switching, and VLAN configuration capabilities that reflect genuine enterprise network operating system behaviour. The AI model serving layer runs via Ollama on a dedicated VM for local model inference, with optional connectivity to the OpenAI API for GPT-based reasoning in configurations where external access is permitted.

4.2 Network Testbed Setup

The network testbed is designed to represent a realistic multi-layer enterprise environment, providing a controlled simulation space in which automation workflows and AI-assisted management operations can be evaluated under conditions that closely mirror real-world network deployments.

4.2.1 Enterprise Network Topology

The enterprise network testbed is structured according to the three-tier enterprise model, comprising a core layer, a distribution layer, and an access layer. This topology was selected because it provides a realistic structure for evaluating network segmentation, inter-VLAN routing, and policy-controlled administrative workflows in a simulated enterprise environment.

The core layer consists of two virtual routers interconnected using OSPF as the dynamic routing protocol, providing redundant paths and fast convergence. The distribution layer introduces Layer 3 switches that perform inter-VLAN routing, while the access layer connects simulated end-hosts, servers, and IoT gateway devices. The entire topology is constructed within GNS3 and incorporates a simulated internet uplink through a NAT cloud node, allowing devices within the testbed to reach external services

where required for AI API calls or package updates.

All routers and switches are addressed according to a structured IP addressing scheme using RFC 1918 private address space, with clearly delineated subnets for each functional zone. Network diagrams are maintained in the project repository to document the logical and physical topology.

In the prototype topology, the perimeter boundary is represented by a firewall component serving as the entry and exit point of the simulated network. Where a FortiGate-based security boundary is described in the implementation context, this refers to the edge filtering policy applied within GNS3, implemented using access control lists on the perimeter router.

4.2.2 VLANs and Segmentation

Network segmentation is implemented through IEEE 802.1Q VLANs to separate traffic between functional device groups. The VLAN architecture defines the following segments, each serving a distinct role in controlling traffic flow and limiting the potential impact of security incidents or device misconfigurations:

- **Management VLAN (VLAN 10):** Used exclusively for SSH administrative access to network devices and management-plane communication. Restricting management traffic to this segment ensures that administrative channels remain isolated from user and IoT traffic.
- **Server VLAN (VLAN 20):** Hosts virtual servers, the AI orchestration stack, and database services. This segment carries the automation and AI reasoning components of the system.
- **IoT VLAN (VLAN 30):** Isolates IoT sensors, actuators, and gateway nodes from the core network, limiting the impact of potential IoT device compromise on broader infrastructure.
- **User VLAN (VLAN 40):** Reserved for simulated end-user workstations and chatbot client access.

VLAN trunking is configured on uplinks between access and distribution switches using 802.1Q encapsulation. Inter-VLAN routing is performed at the distribution layer through switched virtual interfaces (SVIs), with access control lists applied to enforce traffic policies between segments.

4.2.3 Routing and Firewall Configuration

Dynamic routing within the enterprise testbed is provided by OSPF, with all routers participating in a single OSPF area (Area 0) for simplicity. Static default routes are redistributed into OSPF to propagate reachability to the simulated uplink. BGP is not

configured in the current prototype but is identified as a potential future enhancement for multi-site simulation scenarios.

Firewall policy is implemented using access control lists on the perimeter router and reinforced by zone-based policy where Cisco IOS supports it. The following baseline rules are enforced across the prototype testbed:

- Deny unsolicited inbound traffic from the simulated internet zone.
- Permit established sessions returning from internal initiators.
- Restrict IoT VLAN devices from directly initiating connections to the server VLAN except through defined API endpoints.
- Permit SSH only from the management VLAN to device management interfaces.

These rules are provisioned through structured configuration scripts to ensure consistency and auditability across the testbed. Where configuration-management tooling is used to apply these policies, validation should occur before execution and administrator approval should remain mandatory.

4.3 AI and Agent Implementation

The AI reasoning layer is the component responsible for interpreting natural language user requests and determining appropriate automated responses. This section describes the model selection strategy, prompting approach, and intent extraction pipeline used in the prototype implementation.

4.3.1 Model Selection

The AI reasoning layer was designed to support both cloud-connected and locally hosted model configurations. In the prototype context, this includes examples such as GPT-based cloud access and local model hosting through Ollama. Cloud mode was designed to provide higher reasoning quality for complex diagnostic and intent interpretation tasks, while local mode prioritises privacy, offline availability, and reduced operational cost for environments where external API access is restricted.

Model selection is informed by research identifying that LLMs used in network operations must be capable of working with unstructured sources including documentation, command output, and log data. GPT-4-class models are preferred for production-grade diagnostics due to stronger instruction-following behaviour and larger context windows, while smaller open-weight models such as Mistral 7B are considered for development use and latency-constrained scenarios.

Model inference is abstracted behind a unified API gateway within the n8n workflow engine, allowing the underlying model to be swapped without modifying workflow

logic. This decoupling is an intentional architectural feature that improves the long-term maintainability and adaptability of the system as AI model capabilities evolve.

4.3.2 Prompting and Intent Extraction

Prompting strategy is central to the reliability of AI-generated outputs in this system. Rather than relying on zero-shot prompts, the implementation adopts a structured prompting approach that includes three components: a system prompt defining the agent role, a contextual grounding block containing current network state and relevant documentation excerpts, and a user message carrying the operator's natural-language request.

The system prompt instructs the model to act as a network operations assistant with knowledge of Cisco IOS, OSPF, VLANs, and IoT device management. It explicitly instructs the model to avoid fabricating device state, to express uncertainty when context is insufficient, and to format its output as structured JSON when workflow integration is required.

Intent extraction is performed in two stages. In the first stage, the model classifies the user request into a predefined category such as diagnostic query, configuration change, status report, or remediation request. In the second stage, the model extracts the target device, the required action, and any relevant parameters, which are then passed to the corresponding n8n workflow as structured input. This two-stage design was intended to reduce hallucination risk by separating high-level interpretation from parameter extraction, ensuring that downstream automation receives well-formed inputs rather than free-form text.

Where implemented, a retrieval-augmented generation (RAG) component is used to ground model responses in actual network state. Before each AI call, a context assembly module queries the current device inventory, recent telemetry readings, and relevant configuration documentation, and injects this information into the prompt context. The architecture incorporates this retrieval-based grounding step to improve the relevance and accuracy of AI-generated network management responses, aligning with the design approach proposed in the related literature.

4.4 Workflow Automation Implementation

Workflow automation is the operational backbone of the prototype. It translates AI-interpreted user intent into structured, validated, and auditable automation actions against the simulated network environment. This section describes the workflow definitions and the integration mechanisms that connect the orchestration layer with the Python automation scripts.

4.4.1 Workflow Definitions

Workflow automation is implemented using n8n, an open-source workflow orchestration platform hosted as a Docker container within the Ubuntu Server VM on Proxmox. This choice allowed the project to separate request interpretation, workflow coordination, and execution logic within a controllable orchestration environment. Each operational capability of the system is encapsulated as a discrete n8n workflow, allowing independent development, testing, and modification without disrupting other system components.

The following core workflows are defined for the prototype implementation:

- **Device Health Diagnostic Workflow:** Triggered by a user request through the chatbot interface, this workflow issues connectivity tests and interface status commands to a specified device, collects output via SSH, passes the results to the AI reasoning module for summarisation, and returns a natural-language health report to the user.
- **Routing Table Verification Workflow:** Executes routing and adjacency inspection commands against target routers, validates expected adjacencies and prefixes against a stored baseline, and raises an alert if discrepancies are detected.
- **IoT Device Status Workflow:** Queries MQTT broker topics for registered IoT sensor endpoints, retrieves the latest telemetry readings, and compiles a device health summary indicating online status, last-seen timestamp, and sensor value range.
- **Configuration Deployment Workflow:** Accepts a structured configuration intent from the AI layer and maps it to a controlled execution path. Where configuration-management tooling is used, validation occurs before execution and administrator approval remains mandatory. This design establishes a direct workflow path from user request to automated task handling, reducing the need for administrators to interact directly with device CLIs or standalone scripting tools.
- **Periodic Reporting Workflow:** Scheduled to execute at defined intervals, this workflow collects telemetry data, queries device inventory, and generates a structured health report delivered through the chatbot interface.

Each workflow maintains an execution log recording the trigger source, input parameters, execution steps, outputs, and completion status, providing an auditable trail of all automated actions performed by the system.

4.4.2 Integration with Automation Scripts

n8n workflows integrate with the Python automation layer through a combination of HTTPS API calls and subprocess execution. For device communication tasks, n8n invokes

Python scripts via the Execute Command node, passing device identifiers and command parameters as arguments. The Python scripts return structured JSON output, which is parsed by subsequent n8n nodes for further processing or AI submission.

The interaction flow established by this integration layer reflects the layered system architecture described in Chapter 3. As illustrated in Figure 4.2, the system operates as a pipeline in which the user submits a request through the chat interface (Telegram or Web UI), the AI Agent performs intent interpretation and decision support, the n8n Workflow Engine handles workflow validation and orchestration, and the Python Automation Layer executes operations against the GNS3 simulation environment. This separation ensures that AI reasoning and infrastructure execution remain decoupled, improving both controllability and auditability.

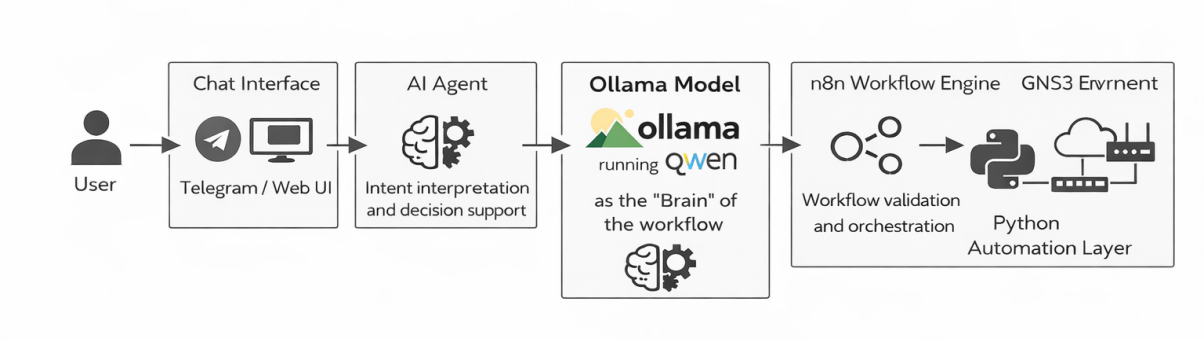


Figure 4.2: System interaction flow showing the layered pipeline from user request through the chat interface, AI agent, n8n workflow engine, and Python automation layer to the GNS3 simulated network environment.

Webhooks are used to trigger workflows from the chatbot interface, with incoming messages carrying a JSON payload containing the parsed user intent, extracted device target, and action type produced by the intent extraction stage. This structured handoff ensures that n8n receives actionable, validated input rather than raw natural-language text, maintaining the integrity of the automation pipeline.

4.5 Python Automation Libraries

The Python automation layer serves as the execution interface between the n8n orchestration layer and the simulated network devices in GNS3. It is responsible for establishing device connections, issuing operational commands, retrieving output, and returning structured results for further processing by the workflow engine and AI module.

4.5.1 Core Prototype Device Access

Python-based device access is implemented using Netmiko and NAPALM as the primary libraries for SSH-based interaction with network devices hosted in GNS3. Structured, scripted device access is essential to the project because it ensures that the automation

layer can interact with network devices using the same protocols and methods as real-world network operations, making the prototype a credible evaluation environment.

Netmiko provides a connection abstraction layer that handles SSH session establishment, command sending, and output retrieval across a wide range of vendor platforms including Cisco IOS. Each device session is instantiated using a connection profile drawn from the encrypted device inventory. NAPALM supplements Netmiko for structured data retrieval, providing normalised getter methods that return device facts, interface status, and routing table information as Python dictionaries rather than raw CLI text. This structured output reduces the need for text parsing and simplifies downstream processing by AI modules and workflow nodes.

For IoT devices, access is implemented over MQTT using the Paho MQTT client library, and over HTTPS using the Requests library for devices exposing REST or RESTCONF interfaces. SNMP polling is supported through PySNMP for legacy devices that do not provide SSH or REST management interfaces, used primarily for availability monitoring and basic interface statistics.

4.5.2 Extended Monitoring and Deployment Support

In addition to the core device interaction libraries, the prototype environment includes support for broader monitoring and optional deployment capabilities. These components may be active in the prototype or may represent architectural extensions available for enabling in expanded deployment scenarios:

- **Telemetry and Monitoring:** Python scripts execute at scheduled intervals via n8n and write collected metrics to a time-series database. Grafana can be connected as a visualisation platform to provide dashboards for interface utilisation, OSPF adjacency state, and IoT sensor readings.
- **Configuration Management Support:** For configuration management and deployment, the prototype proposes or optionally incorporates configuration-management tooling such as Ansible, particularly where controlled playbook-driven changes are required. Where retained, this capability should be supported by actual prototype evidence, with the check-mode dry-run validation step used before any changes are committed to execution.

4.6 Security and Access Control

This section describes the security measures incorporated into the prototype. The security content is organised into two subsections. The first covers security controls that are directly demonstrable within the prototype environment. The second addresses security considerations that are architecturally relevant but more appropriate to real-world deployment contexts rather than the current prototype scope.

4.6.1 Prototype-Level Security Controls

The following security controls are implemented within the prototype environment and reflect the actual configuration of the system as tested and demonstrated.

Interface and Access Restriction

Access to the administrative interface is restricted to authorised administrators only. The system operates under an administrator-only usage model, where only authenticated team members may submit operational requests or trigger workflow execution. Role-based access control distinguishes between administrative users, who may trigger configuration changes and view all device data, and read-only users, who may only execute diagnostic queries and view reports. Authentication occurs before reaching the application layer, providing a baseline level of access protection.

Administrator Approval for Write Operations

Configuration-related workflow actions are gated behind an explicit administrator approval step before execution. This confirmation mechanism is enforced for all workflows categorised as write operations, while read-only diagnostics may proceed without explicit approval. This design reflects the human-in-the-loop validation principle described earlier, ensuring that potentially disruptive changes are reviewed before being applied to the simulated network.

SSH-Based Device Access and Host Key Verification

SSH is used exclusively for device management access within the prototype. Host key verification is enabled to prevent man-in-the-middle exposure within the testbed network. Device connections are established using credentials drawn from the protected device inventory rather than plain-text configuration files, ensuring that sensitive authentication data is not exposed in workflow definitions, scripts, or version control repositories.

Internal Network Confinement

Communication between the n8n orchestration engine, the Python automation layer, and the database backend is confined to the internal network bridge where possible, limiting exposure to the host network interface. This confinement ensures that core automation services are not directly reachable from external networks under normal prototype operating conditions.

Lab-Level HTTPS Exposure

The chatbot web interface is served over HTTPS through a reverse proxy, with self-signed certificates used for isolated laboratory environments. This ensures that communications

within the prototype remain encrypted in transit, protecting authentication credentials and automation commands during development and testing.

VLAN-Based Network Isolation

Network segmentation enforced through the VLAN architecture described in Section 4.2 provides an additional layer of isolation within the prototype. IoT device traffic cannot reach management or server segments except through explicitly permitted access control paths. This is one of the clearest security controls demonstrable within the prototype topology, as the VLAN structure is directly observable and testable within the GNS3 simulation environment.

4.6.2 Security Considerations for Real-World Deployment

The following security concepts are architecturally relevant to the system but reflect enterprise-grade practices that extend beyond the current prototype scope. They are included here to acknowledge the security thinking that informed the design and to guide future development or production deployment of the system.

Dedicated Secrets Management Backend

For production deployments, device credentials for SSH access and API keys for external services should be managed through a dedicated secrets management backend such as HashiCorp Vault or an equivalent solution. This would support credential rotation, runtime retrieval via a secrets API, and enforcement that no sensitive data is stored in plain-text configuration files, workflow definitions, or version control repositories. In the current prototype, the principle of protected credential handling is applied, but a full enterprise-grade secrets rotation infrastructure is identified as a deployment-stage requirement.

Production TLS Certificate Management

For internet-accessible deployments, TLS certificate management should be handled through a production-grade certificate authority such as Let's Encrypt, with automated renewal and monitoring. In the prototype, self-signed certificates are used for isolated laboratory environments, which is appropriate for the current scope. Transitioning to publicly trusted certificates would be necessary if the system were to be exposed to external users or deployed on a publicly reachable host.

External API Key Governance

In a production environment, API keys for cloud AI services and any other external integrations should be subject to formal governance processes including key rotation

schedules, access logging, and revocation procedures. In the prototype, keys are stored as environment variables injected at container startup, which prevents hardcoding in application code but does not provide the full audit trail appropriate for enterprise governance.

MQTT Broker Security for IoT Deployment

Where IoT telemetry is integrated through MQTT beyond the laboratory environment, broker-level authentication and TLS encryption should be enabled on the MQTT broker listener to protect sensor data in transit. Client authentication should require username and password for all connections. In the prototype, MQTT communication is configured within the controlled VLAN-isolated testbed environment; the additional broker security controls described here are appropriate for deployments where IoT endpoints communicate across less trusted network boundaries.

Enterprise Jump-Host and Cross-Boundary Access Policy

For enterprise-scale deployments where management access crosses network trust boundaries, jump-host infrastructure and formalised access policies should be established. This would ensure that administrative SSH sessions are routed through controlled access points with full session logging, rather than directly from the orchestration layer to managed devices. This is identified as a deployment-stage architecture enhancement rather than a prototype-level requirement.

Chapter Summary

This chapter has described the implementation of the AI-driven network and IoT management prototype across five technical areas: the virtualized development environment, the enterprise network testbed, the AI and agent implementation, the workflow automation layer, and the security framework.

The Proxmox-based virtualization environment provides the isolation and reproducibility required for systematic prototype evaluation. The GNS3 network testbed replicates a realistic multi-tier enterprise topology with VLAN segmentation and OSPF-based routing, offering a credible evaluation environment for automation scenarios. The AI reasoning layer supports structured intent extraction through a two-stage prompting pipeline, with model inference abstracted behind a unified gateway to support future model substitution. The n8n orchestration layer translates AI-interpreted intent into controlled, auditable workflow executions, with explicit approval gates enforced for all write operations. The Python automation layer provides direct device interaction through industry-standard libraries, forming the execution interface between workflows and simulated network devices.

The security framework distinguishes clearly between controls demonstrated within the prototype and considerations relevant to production deployment. This distinction ensures that the chapter accurately represents the prototype's scope while acknowledging the broader security architecture that would be required in a real-world operational context.

Chapter 5

Experimental Evaluation and Results

This chapter presents the experimental evaluation results of the implemented AI-assisted infrastructure management prototype. The purpose of the chapter is to show how the workflows performed in practical test scenarios, using Telegram-based interaction, n8n workflow automation, system evidence, network policy evidence, and human-in-the-loop control.

The evaluation is separated into two main layers. The Systems Layer Evaluation focuses on systems infrastructure administration and systems security monitoring. The Network Layer Evaluation focuses on employee network issue reporting and administrator network operations. This structure makes it possible to evaluate both categories of the project separately while still showing how they work together as parts of one broader infrastructure support system.

5.1 Systems Layer Evaluation

5.1.1 Systems Infrastructure Administration Evaluation

Introduction

This section presents the experimental evaluation results for the systems infrastructure administration component of the project. The evaluation demonstrates how an AI-assisted, human-in-the-loop approach can support systems administration tasks in a company-like infrastructure environment.

The implemented system uses a Telegram-based administrator interface connected to n8n workflows. The main communication workflow is the Proxmox Admin Bot, while the OpenClaw Proxmox Orchestrator operates as a central sub-workflow responsible for routing systems administration requests to specialist sub-workflows.

The purpose of this evaluation is to demonstrate that AI-assisted workflow automation can help a systems administrator perform common infrastructure management tasks more safely, consistently, and quickly. The tested operations include safe patching, resource diagnostics, unreachable VM triage, idle VM cleanup, and IP address discovery.

Unlike a fully autonomous system, this implementation preserves human control for risky operations. Human approval is required before actions such as patching, rollback, rebooting, stopping, or destructive maintenance are executed.

Evaluation Environment Overview

The evaluation was conducted in a Proxmox-based virtualized laboratory environment. The environment contained multiple virtual machines representing common systems administration assets.

Table 5.1: Systems infrastructure administration evaluation environment.

Item	Description
Main workflow	Proxmox Admin Bot
Main communication channel	Telegram
Orchestration sub-workflow	OpenClaw Proxmox Orchestrator
Automation platform	n8n
Virtualization platform	Proxmox VE
Primary test VM	UbuntuTest
Test VMID	105
Human-in-the-loop control	Confirmation prompts before risky actions

The Proxmox Admin Bot receives administrator messages through Telegram. It then normalizes the request, handles confirmation logic where necessary, and calls the OpenClaw Proxmox Orchestrator as a sub-workflow. The orchestrator routes the request to the correct specialist sub-workflow.

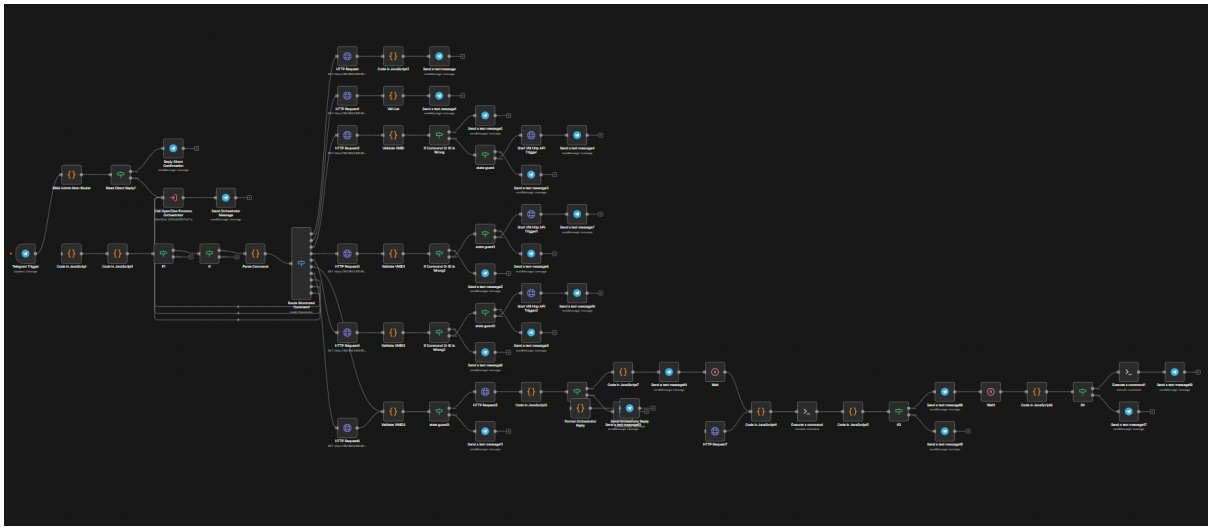


Figure 5.1: Evaluation environment and Proxmox Admin Bot workflow overview.

Figure 5.1 shows the main Proxmox Admin Bot workflow in n8n. The workflow begins with the Telegram trigger, prepares and parses the administrator message, checks whether the request requires direct confirmation, and then routes structured commands to the relevant operation branch. This figure is important because it shows that the Telegram bot is not a simple chat interface; it is connected to a controlled workflow that performs parsing, routing, confirmation handling, and response delivery.

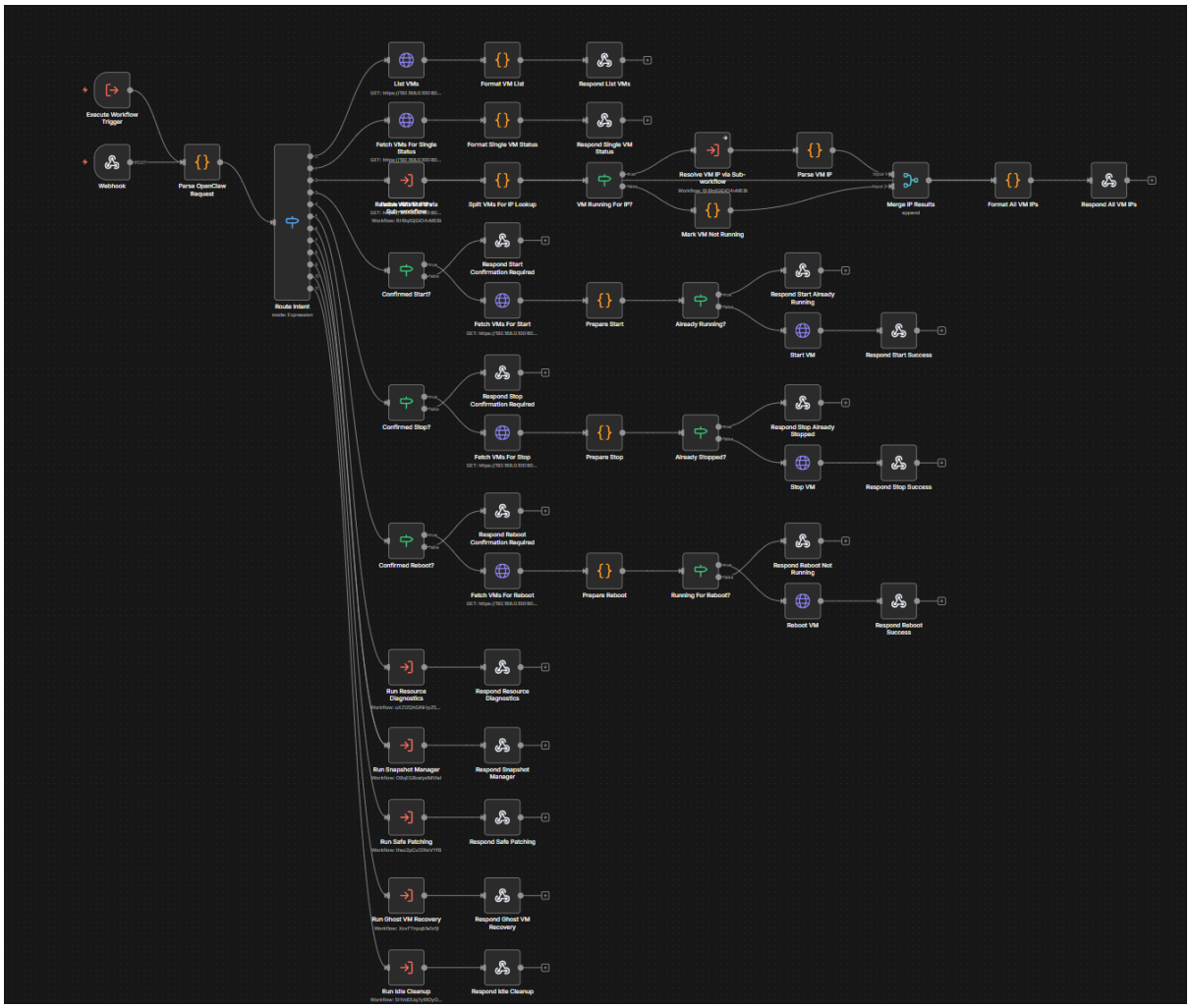


Figure 5.2: OpenClaw Proxmox Orchestrator sub-workflow overview.

Figure 5.2 shows the OpenClaw Proxmox Orchestrator sub-workflow. The orchestrator receives structured requests from the main bot and routes them to specialist branches such as VM listing, IP resolution, start/stop/reboot control, resource diagnostics, snapshot management, safe patching, ghost VM recovery, and idle cleanup. This separation makes the system more maintainable because the main communication workflow can remain focused on Telegram interaction while the orchestrator handles infrastructure-specific routing.

System Input, Processing, and Output

Input

The system receives natural-language Telegram messages from a systems administrator. The administrator does not need to use only strict commands. Example inputs include:

- `Ubuntu Test is slow`
- `safely update Ubuntu Test`
- `People cannot access Ubuntu Test`
- `find idle VMs and suggest cleanup`
- `Get me the ip addresses of all the machines`
- `Get me detailed information of VMs`

Processing

After receiving a Telegram message, the Proxmox Admin Bot performs initial processing. The main processing sequence is as follows:

1. The Telegram message is received by the Proxmox Admin Bot.
2. The request is normalized into structured fields.
3. The workflow determines whether the request is read-only or requires approval.
4. If approval is required, a human confirmation prompt is returned.
5. If the request is approved or read-only, the OpenClaw Proxmox Orchestrator is called as a sub-workflow.
6. The orchestrator routes the request to the relevant specialist workflow.
7. The specialist workflow interacts with Proxmox or the target VM.
8. The result is formatted and returned to the administrator through Telegram.

Output

The system produces Telegram-based administrative responses. These include:

- formatted VM IP summaries;
- resource diagnostic reports;
- safe patching confirmation prompts;
- snapshot and health check evidence;
- recovery recommendations;
- idle VM cleanup suggestions.

The output is designed to be readable by a systems administrator and includes headings, VM identifiers, state information, and recommended next steps.

General Operational Flow

The general operational flow of the system is shown below:

1. A systems administrator sends a Telegram message.
2. Proxmox Admin Bot receives the message.
3. The message is interpreted and normalized.
4. If the action is risky, the bot asks for human confirmation.
5. The OpenClaw Proxmox Orchestrator is called as a sub-workflow.
6. The orchestrator invokes the appropriate specialist sub-workflow.
7. Proxmox or the guest VM is queried or modified.
8. The result is formatted and returned to Telegram.

This flow demonstrates a modular, event-driven, and human-supervised approach to systems infrastructure administration.

Figure 5.3 summarizes the complete systems administration design at a conceptual level. It shows the administrator sending requests through Telegram, the Proxmox Admin Bot receiving the request, the AI interpretation and routing layer mapping the request to an intent, and the OpenClaw Proxmox Orchestrator forwarding the request to the correct specialist module. The diagram also distinguishes read-only operations from approval-gated actions, which reflects the system's safety-first design for patching, rollback, reboot, stop, and destructive operations.

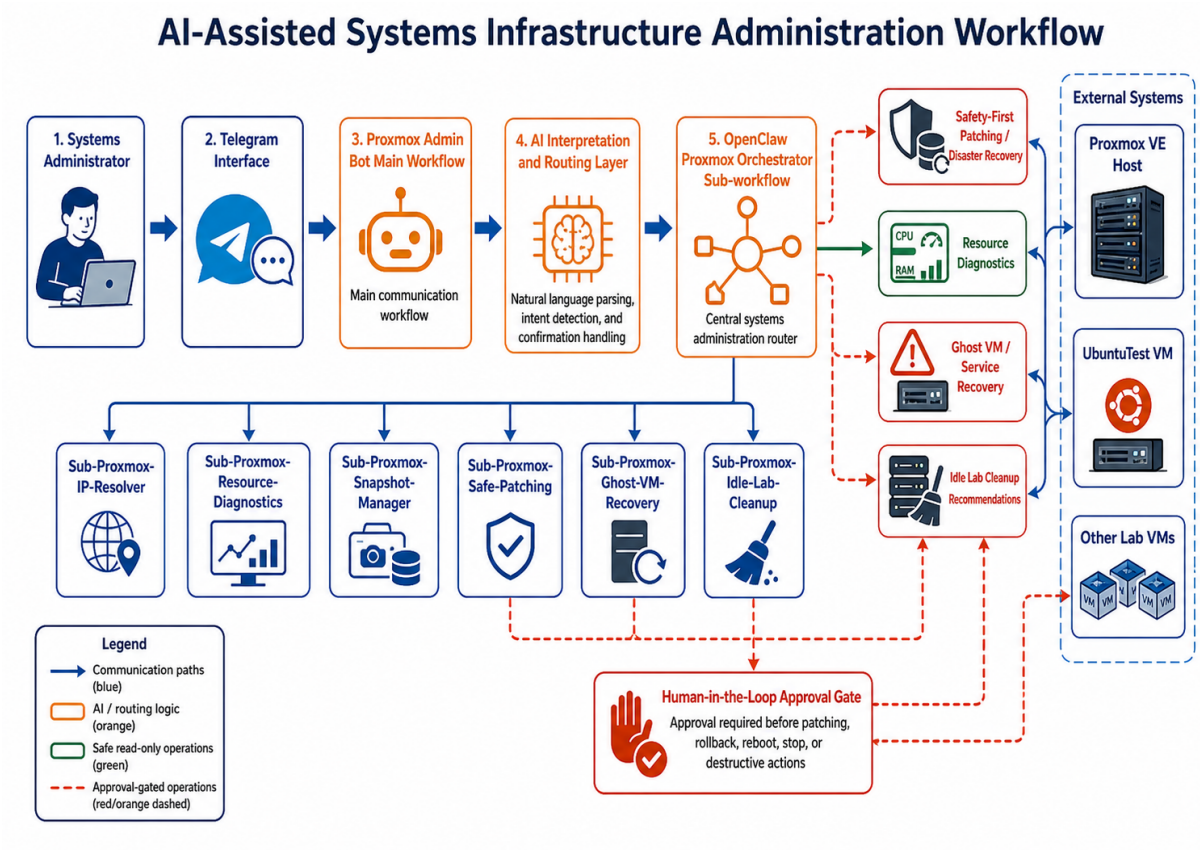


Figure 5.3: AI-assisted systems infrastructure administration workflow.

Main Functional Modules

Telegram Communication Module

The Telegram communication module is implemented inside the Proxmox Admin Bot workflow. It serves as the main entry point for the administrator. This module receives messages, sends confirmation prompts, and returns final formatted results.

This design allows the systems administrator to interact with the infrastructure through a familiar chat interface instead of manually logging into multiple systems.

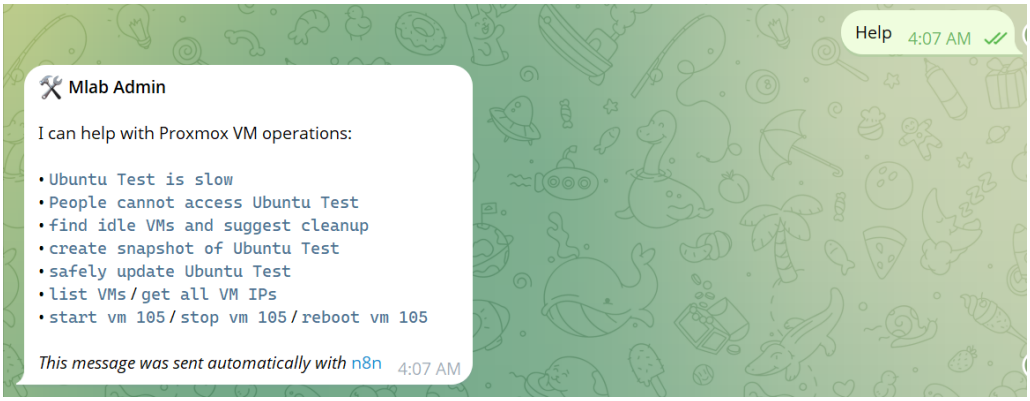


Figure 5.4: Telegram message intake and main bot communication.

Figure 5.4 shows the Telegram help response returned by the administrator bot. The output lists the supported Proxmox VM operations, including slow VM diagnostics, inaccessible VM recovery, idle VM cleanup, snapshot creation, safe updating, IP listing, and VM start/stop/reboot commands. This confirms that the bot exposes the available infrastructure operations through a familiar chat interface and guides the administrator toward valid requests.

AI Interpretation and Routing Module

The routing module interprets administrator messages and maps them to infrastructure intents. These intents include resource diagnostics, safe patching, IP discovery, recovery triage, idle cleanup, and VM control.

The workflow can use an n8n AI Agent connected to a language model to make interaction more natural. However, the system still uses deterministic parsing and structured workflow routing to preserve reliability during infrastructure operations.

This module improves usability while keeping operational execution controlled and traceable.

Human-in-the-Loop Approval Module

The human-in-the-loop module prevents risky operations from executing automatically. When an administrator requests an operation such as safe patching, rollback, reboot, stop, or destructive maintenance, the bot first asks for confirmation.

For example, when the administrator sends `safe update Ubuntu Test`, the bot returns a confirmation message and waits for the administrator to reply `yes`. Only after approval does the workflow continue.

This module reduces the risk of accidental service disruption and supports accountability in AI-assisted systems management.

Figure 5.5 shows the complete safe patching interaction. The administrator requests a safe update for UbuntuTest, and the bot first asks for confirmation instead of immediately executing the update. After the administrator replies yes, the workflow creates a pre-change snapshot, runs the update, performs health checks, and returns rollback guidance. This figure demonstrates the human-in-the-loop control used for risky systems operations.

OpenClaw Proxmox Orchestrator Module

The OpenClaw Proxmox Orchestrator is the central sub-workflow responsible for systems administration routing. It receives structured requests from the Proxmox Admin Bot and calls the appropriate specialist sub-workflow.

The orchestrator supports the following major operations:

- VM status and detailed VM information;

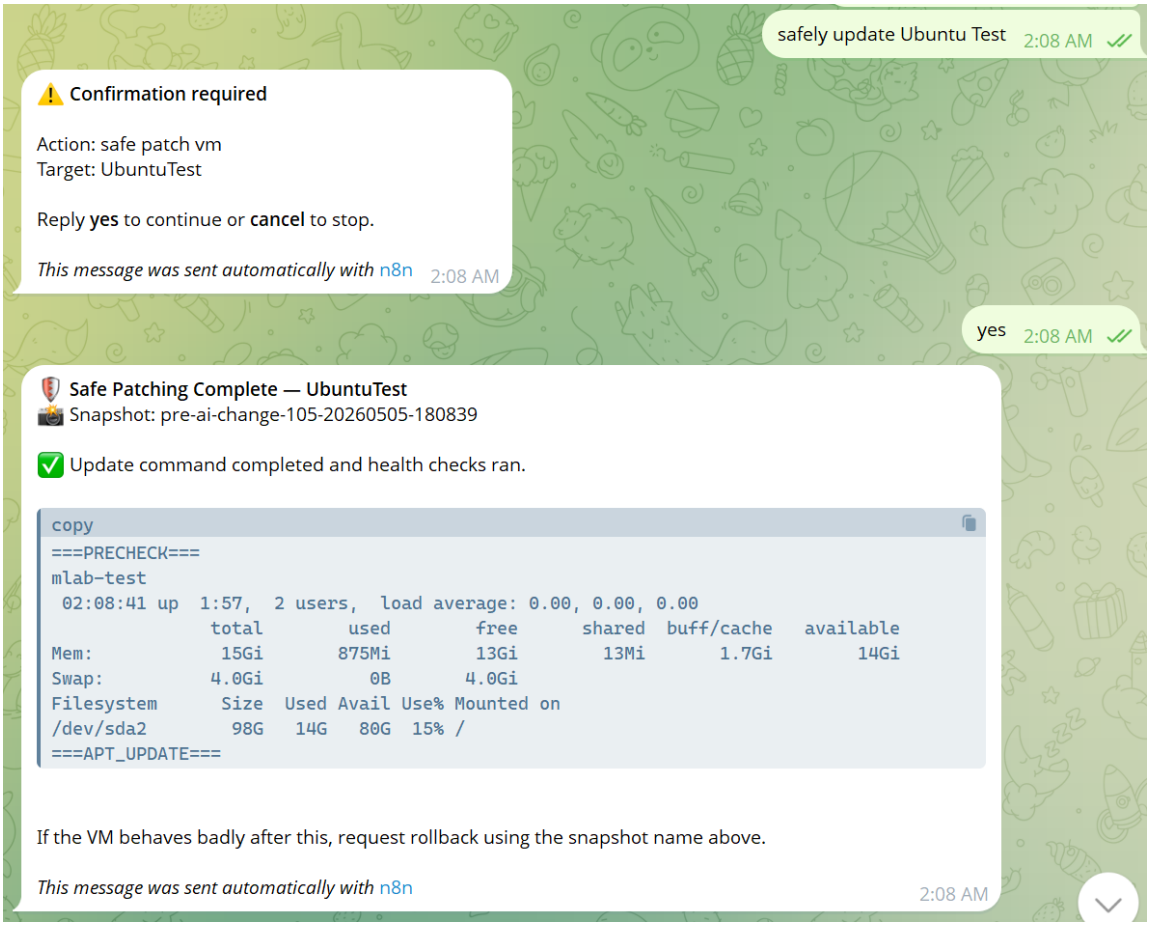


Figure 5.5: Safe patching request, confirmation prompt, administrator approval, snapshot creation, and health-check result.

- VM IP address discovery;
- resource diagnostics;
- safe patching;
- snapshot creation and rollback support;
- unreachable VM recovery triage;
- idle VM cleanup recommendations.

This module acts as the central logic layer of the systems administration automation design. The orchestrator routing branches and sub-workflow calls are shown in Figure 5.2.

Proxmox IP Resolver Module

The IP resolver module discovers VM IP addresses without relying on QEMU Guest Agent. This is important because the evaluated VMs do not all have the guest agent installed.

The resolver uses agentless methods such as:

- Proxmox VM inventory;
- VM MAC address extraction;
- bridge neighbor table lookup;
- `arp-scan` results.

This allows the system to identify VM IP addresses even when addresses change dynamically.

Figure 5.6 shows the formatted IP discovery output returned through Telegram. The response lists each VM with its VMID, name, running state, Proxmox node, IPv4 address, MAC address, and IP discovery source. Stopped VMs are explicitly marked as stopped, which avoids misleading empty IP fields. This output demonstrates that the workflow can perform agentless IP discovery using VM metadata, MAC addresses, bridge neighbor information, and ARP-based evidence.

Evaluation Scenarios and Results

Scenario A: Safety-First Patching and Disaster Recovery

Scenario description. This scenario evaluates whether the system can support low-risk operating system maintenance. In a company environment, patching a server can be risky because updates may cause application or service failure. The workflow therefore creates a safety point and requires human approval before proceeding.

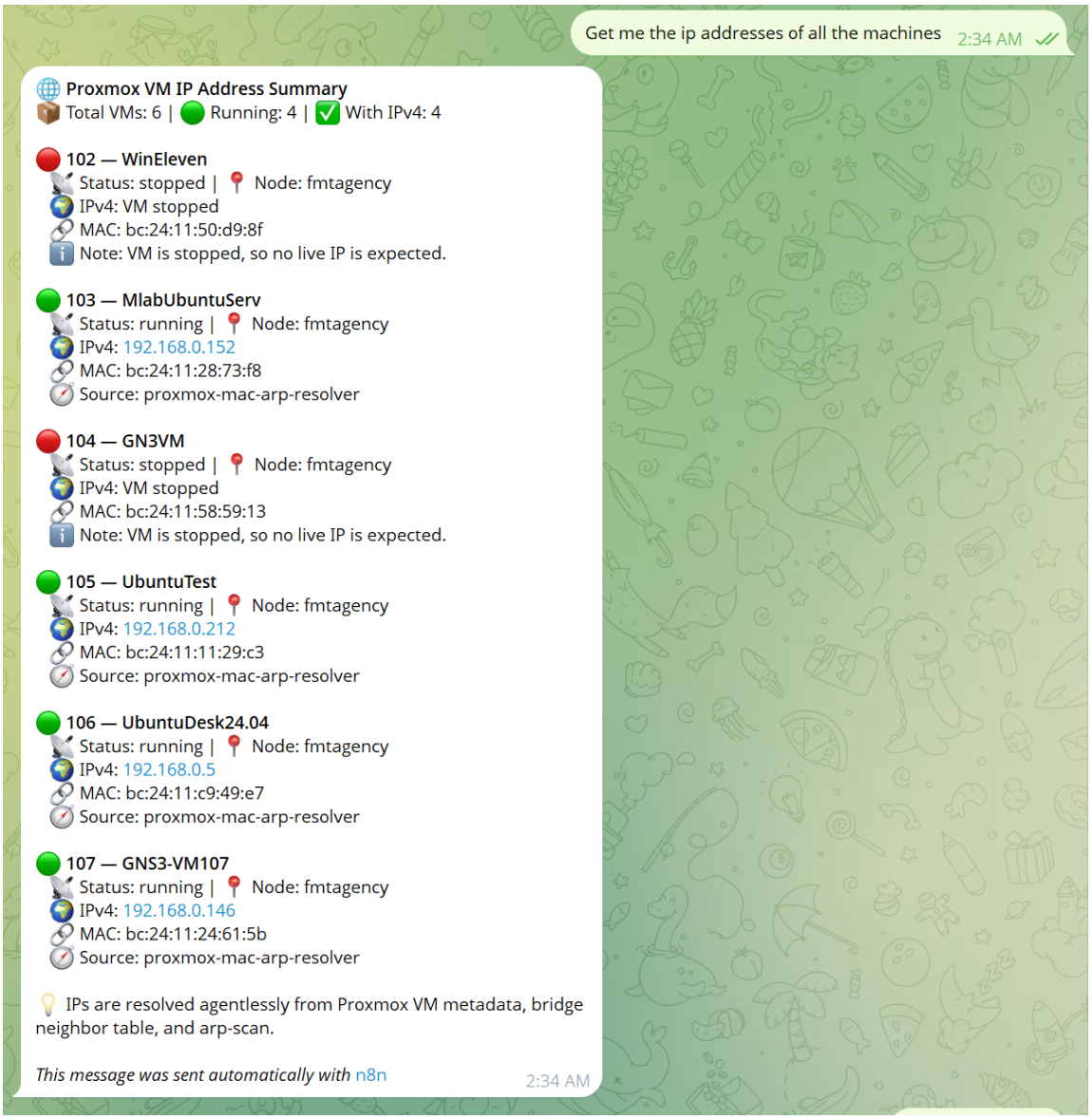


Figure 5.6: Formatted VM IP address summary.

Test input. The test input was `safe update Ubuntu Test` or `safely update Ubuntu Test`.

Expected behaviour. The system should identify the target VM as UbuntuTest, ask for administrator approval, create a Proxmox snapshot before patching, run the update workflow after approval, return health check results, and provide rollback guidance.

Observed result. The bot returned a confirmation prompt before executing the operation. After approval, the system created a safety snapshot and returned a formatted safe patching report. The report included the snapshot name, update status, basic health check output, and rollback guidance. The complete Telegram evidence is shown in Figure 5.5.

Evaluation. The result shows that the system successfully applies human-in-the-loop control to a risky systems administration task. Instead of allowing the AI workflow to patch a VM directly, the administrator remains responsible for approving the operation. This supports safer infrastructure management and provides a recovery point through snapshot creation.

Scenario B: Resource Diagnostics for a Slow VM

Scenario description. This scenario evaluates whether the system can assist a systems administrator in diagnosing VM performance issues. In a real company environment, users may report that a system is slow without knowing whether the issue is caused by CPU, memory, disk, network, or application processes.

Test input. The test input was `Ubuntu Test is slow`.

Expected behaviour. The system should resolve the target VM, collect Proxmox resource metrics, identify the VM IP address, SSH into the guest operating system, collect uptime, memory, disk, top CPU processes, top memory processes, and failed services, and return a readable diagnostic summary and recommendation.

Observed result. The bot returned a resource diagnostics report for UbuntuTest. The report included Proxmox metrics such as RAM, CPU, disk size, network traffic, and uptime. It also included guest OS evidence collected through SSH, including memory usage, disk usage, top processes, and failed services. The full diagnostic output is shown in Figure 5.7.

Figure 5.7 shows the resource diagnostics result for the request "Ubuntu Test is slow". The response combines Proxmox-level information, such as VMID, node, state, IP address, RAM usage, CPU usage, disk size, network traffic, and uptime, with guest operating system evidence collected through SSH. The guest snapshot includes hostname, uptime, memory, disk usage, top CPU processes, top memory processes, failed services, and a recommendation. This combined output reduces the amount of manual checking normally required during initial performance triage.

Evaluation. The result demonstrates that AI-assisted workflow automation can reduce the time required for initial performance triage. Instead of manually checking Prox-

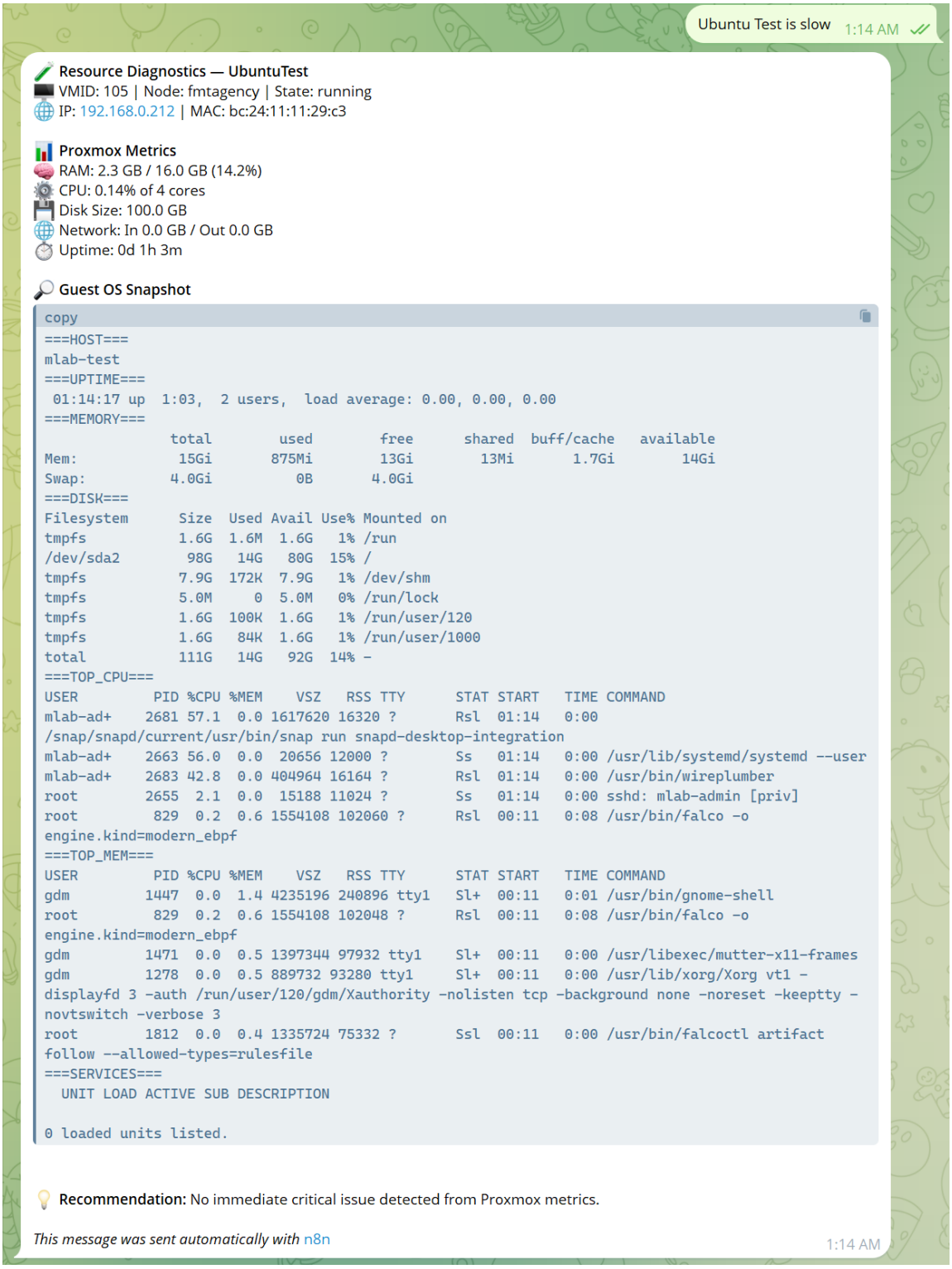


Figure 5.7: Slow VM diagnostic request, Proxmox metrics, guest OS snapshot, and recommendation output.

mox, resolving the VM IP address, and logging into the guest system, the administrator receives a combined diagnostic report through Telegram.

Scenario C: Ghost VM or Unreachable Service Recovery

Scenario description. This scenario evaluates whether the system can help a systems administrator investigate a service or VM that users cannot access. The goal is not to immediately reboot the VM, but to follow a least-disruptive recovery process.

Test input. The test input was `People cannot access Ubuntu Test`.

Expected behaviour. The system should resolve the VM identity, check whether the VM is running, resolve the current IP address, return triage information, and recommend least-disruptive next steps before rebooting.

Observed result. The bot returned a recovery check showing the VMID, running state, and IP address. The workflow recommended a step-by-step escalation path, including ping, SSH, service checks, service restart, and reboot only after approval. The request and recovery output are shown together in Figure 5.8.

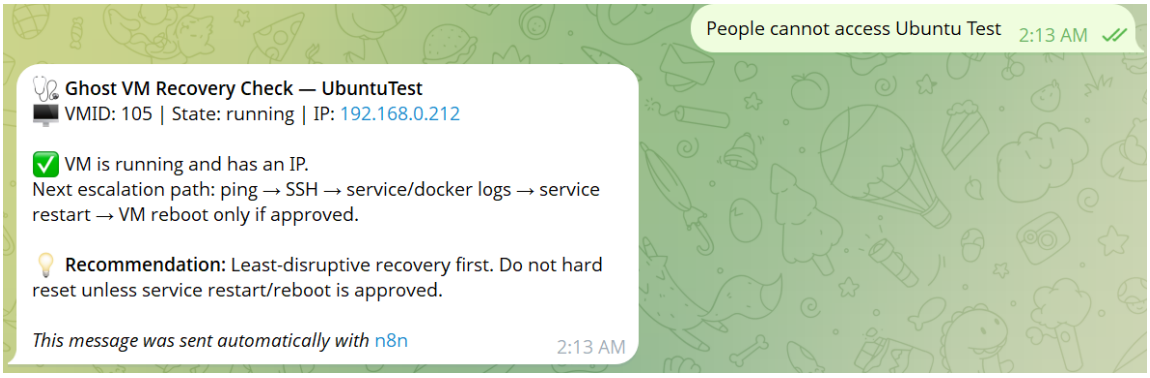


Figure 5.8: Unreachable VM recovery request, VM state, IP resolution, and least-disruptive recovery recommendation.

Figure 5.8 shows the recovery triage output for a reported access problem. The bot identifies UbuntuTest, confirms that VMID 105 is running, resolves its current IP address, and recommends a least-disruptive escalation path. The recommendation begins with ping and SSH checks, then moves to service or Docker log inspection, service restart, and VM reboot only after approval. This supports controlled incident response rather than immediate disruptive recovery actions.

Evaluation. The result shows that the system supports operational discipline during incident response. Instead of immediately rebooting the VM, the system provides a structured recovery path that prioritizes low-impact checks first.

Scenario E: Idle Lab Cleanup Recommendations

Scenario description. This scenario evaluates whether the system can identify potentially idle or unused lab VMs and recommend cleanup actions. In infrastructure

administration, unused VMs can waste storage, CPU allocation, and memory resources.

Test input. The test input was `find idle VMs and suggest cleanup`.

Expected behaviour. The system should retrieve VM inventory, identify stopped or low-use VMs, produce cleanup recommendations, and avoid destructive action unless separately approved.

Observed result. The bot returned a list of candidate VMs for review, including stopped VMs. The response clearly indicated that the output was recommendation-only and that stopping or removing VMs requires separate approval. The combined request and output are shown in Figure 5.9.

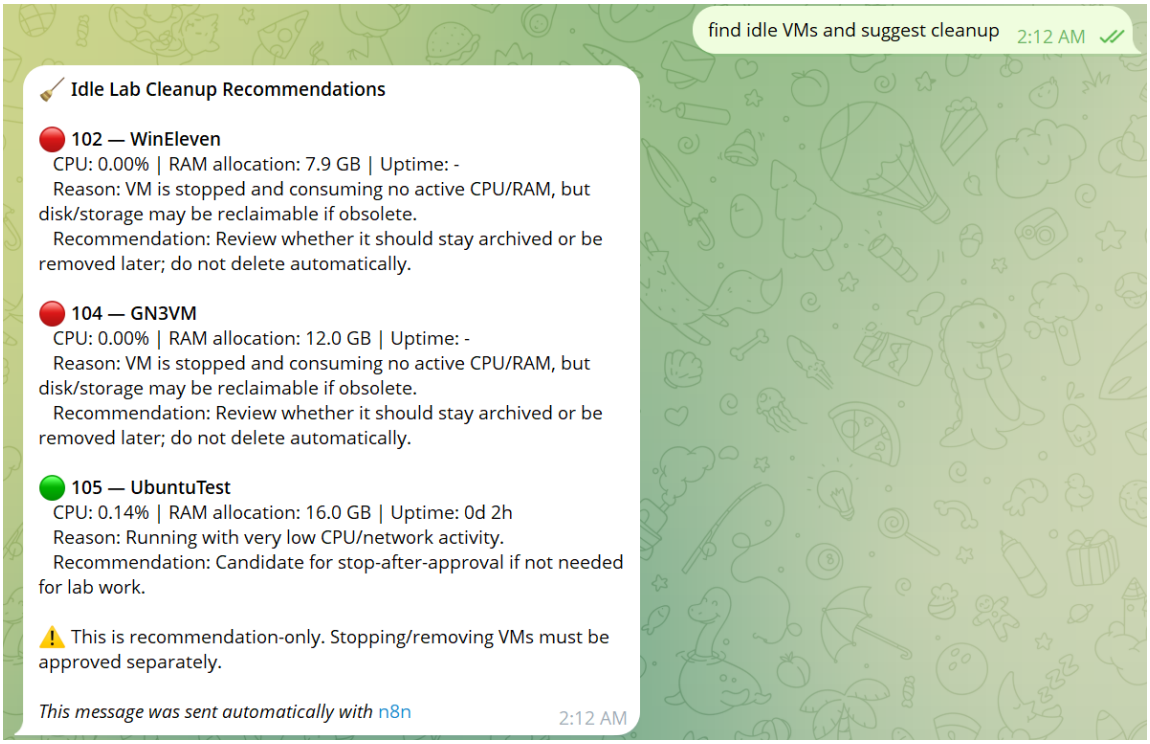


Figure 5.9: Idle VM cleanup request and recommendation-only cleanup output.

Figure 5.9 shows the idle lab cleanup recommendation output. The workflow identifies stopped or low-use VMs such as WinEleven, GN3VM, and UbuntuTest, then explains why each VM may be a cleanup candidate. The response is explicitly recommendation-only and states that stopping or removing VMs requires separate approval. This demonstrates how the bot can support capacity review without performing destructive cleanup automatically.

Evaluation. The result demonstrates how AI-assisted workflow automation can support capacity review without making unsafe automatic changes. The workflow helps the administrator identify possible cleanup opportunities while keeping final action under human control.

Supporting Result: VM IP Address Discovery

Although not one of the four main demonstration scenarios, IP discovery is an important supporting capability. Many administrative operations require the system to know the current IP address of a VM.

Test input. The test input was "Get me the ip addresses of all the machines".

Observed result. The workflow returned a formatted IP address summary showing each VM, its status, node, IPv4 address, MAC address, and IP source. Stopped VMs were clearly marked as stopped instead of showing confusing null values. The formatted output is shown in Figure 5.6.

Evaluation. This result demonstrates that the system can perform agentless VM discovery. This is useful where QEMU Guest Agent is not installed or cannot be relied upon.

Control Features and Risks

Control Features

The implementation includes several control features:

- human approval before risky actions;
- Proxmox snapshot creation before patching;
- rollback guidance after patching;
- read-only diagnostics for performance issues;
- non-destructive idle cleanup recommendations;
- structured Telegram responses for administrator review;
- modular sub-workflows for maintainability.

Risks

Possible risks include:

- incorrect AI interpretation of ambiguous natural-language requests;
- failed SSH access to guest VMs;
- incomplete IP discovery if the VM is not visible in ARP or neighbor tables;
- Telegram or n8n workflow failure;
- administrator approving a risky operation without reviewing the context;

- output formatting errors if raw JSON is not converted into Telegram-ready text.

The implementation reduces these risks by using deterministic workflow routing, approval prompts, formatted outputs, and modular sub-workflows.

Summary of Findings

The systems infrastructure administration evaluation shows that the implemented system can support practical systems administration tasks using AI-assisted workflow automation and human-in-the-loop control.

The strongest findings are:

- natural-language Telegram interaction makes administration easier;
- the workflow can resolve VM identity and IP addresses dynamically;
- resource diagnostics combine Proxmox metrics with guest OS evidence;
- risky actions are protected by approval prompts;
- snapshot creation improves safety before patching;
- idle cleanup recommendations support capacity management without destructive automation;
- modular sub-workflows improve maintainability and separation of responsibility.

The evaluation therefore supports the project argument that AI can assist network and systems management when it is combined with structured automation, human oversight, and clear operational controls.

Conclusion

This section presented the experimental evaluation results for the systems infrastructure administration component of the project. The evaluation demonstrated how a systems administrator can use a Telegram-based AI-assisted workflow to perform infrastructure operations in a Proxmox environment.

The tested scenarios showed that the system can support safe patching, VM performance diagnostics, unreachable VM recovery, and idle VM cleanup recommendations. The system also provides supporting functions such as IP discovery and detailed VM information.

Overall, the results show that the proposed AI-assisted, human-in-the-loop approach can improve systems administration by reducing manual effort, improving response consistency, and preserving safety for risky operations.

5.1.2 Systems Security Monitoring Evaluation

Introduction

This subsection presents the systems security monitoring evaluation contributed as part of the systems layer testing. The evaluated workflow is an AI Security Guard for Ubuntu workplace monitoring and policy violation detection. Its purpose is to demonstrate how a company-managed Ubuntu endpoint can be monitored for unauthorized user behavior, policy-warning activity, and repeated attempts to perform restricted actions.

In this evaluation, the Ubuntu machine `mlab-test` represents a company-managed endpoint used by multiple test users. The users are organized into company-like groups, similar to a simplified Active Directory environment. These groups include administrators, normal employees, interns, contractors, and policy violators.

The security workflow combines Linux security logging, audit rules, custom policy-warning scripts, n8n automation, SSH-based log collection, Telegram notifications, and an AI analysis layer. The goal is not only to detect policy violations, but also to help the administrator understand the evidence quickly in plain English. Instead of manually reading raw Ubuntu logs, the administrator can receive scheduled alerts or ask the Telegram bot questions such as "give me a report for the last 1 hour" or "why was employee2 flagged?"

Use Case Description

The use case focuses on workplace endpoint monitoring. Users are allowed to use company devices, but are not allowed to perform certain actions such as installing unauthorized software, running restricted commands, or attempting administrative actions without permission.

The Ubuntu machine is configured with the following test users:

- `mlab-admin`: administrator user with sudo privileges;
- `employee1`: trusted employee with sudo privileges;
- `employee2`: normal employee without sudo privileges;
- `employee3`: normal employee without sudo privileges;
- `intern1`: restricted intern account;
- `contractor1`: restricted contractor account.

The users are organized into company-style groups:

- `company-admins`;
- `company-employees`;

- `company-interns`;
- `company-contractors`;
- `policy-violators`.

The security layer monitors repeated restricted software installation attempts, unauthorized sudo usage, execution of monitored administrative tools, policy-warning events generated by the company warning script, and user activity over selected timeframes such as today, the last 1 hour, or the last 5 days.

A threshold of five policy violations is used. If a user has fewer than five violations, the activity is treated as a warning. If the user reaches five or more violations, the system marks the user as needing administrator review and the scheduled workflow sends a Telegram alert.

Security Workflow Operation

The workflow supports two main types of interaction: scheduled monitoring and Telegram report requests.

Scheduled Monitoring Flow

The scheduled monitoring branch is used for automatic security checks. In the current implementation, the workflow runs at a fixed interval and prepares a monitoring request with a default timeframe, such as the last 1 hour.

The workflow connects to `mlab-test` through SSH and executes the log collector script:

```
sudo -n /usr/local/sbin/collect-company-logs.sh "last 1 hour"
```

The collector gathers relevant system information, user information, policy warnings, audit events, and a structured policy violation summary. The structured summary includes fields such as `USER`, `status`, `policy_warnings`, `software_install`, and `total_violations`. The workflow parses this structured output into JSON containing per-user violation counts, warning users, users needing administrator review, attempted commands, total policy violations, and alert status.

If no user has reached the threshold, the scheduled branch stops silently. If one or more users have reached the threshold, the workflow builds a Telegram alert and sends it to the administrator. This makes the scheduled monitoring branch reliable because the alert decision is based on deterministic rules rather than AI guesses.

Telegram Report Request Flow

The Telegram report branch begins when the administrator sends a message to the Telegram bot. The Telegram Trigger receives the message and passes it to the Normalize Telegram Request node. This node extracts the chat ID, original request, timeframe, requested user where present, request type, and report intent.

For example, if the administrator sends "give me a report on employee2s activity today", the normalizer prepares a structured request indicating chat mode, host `mlab-test`, timeframe `today`, request type `user_warnings`, user filter `employee2`, and report-request status.

The workflow then runs the collector script using the requested timeframe. The Build Chat Security Context node parses the output and prepares structured evidence for the AI Risk Analyst node. The AI Risk Analyst does not decide the threshold. Instead, it explains the evidence in plain English and can answer which users need admin review, why a user was flagged, and what activity was observed during a selected timeframe.

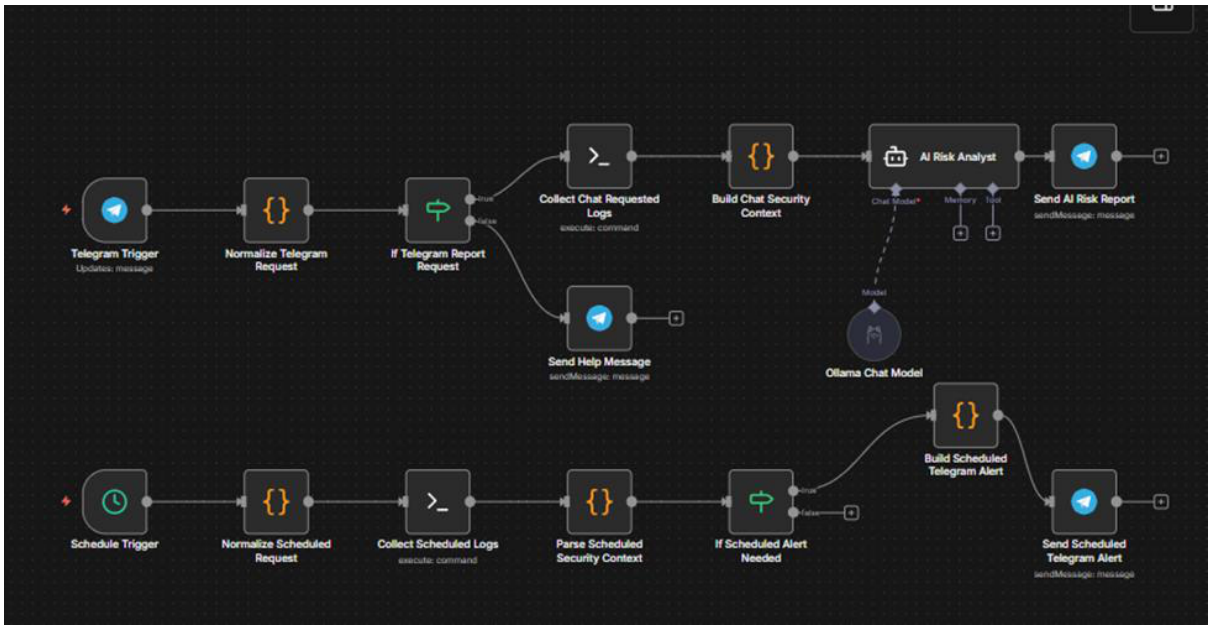


Figure 5.10: AI Security Guard workflow design.

Security Testing Scenarios and Results

The testing category for this use case is systems administration and security monitoring automation. The tests focus on validating whether the security workflow can collect Ubuntu logs through SSH, detect policy-warning activity, separate warning-level users from users needing administrator review, apply timeframe filtering correctly, send scheduled Telegram alerts, and answer administrator security questions through the AI analyst.

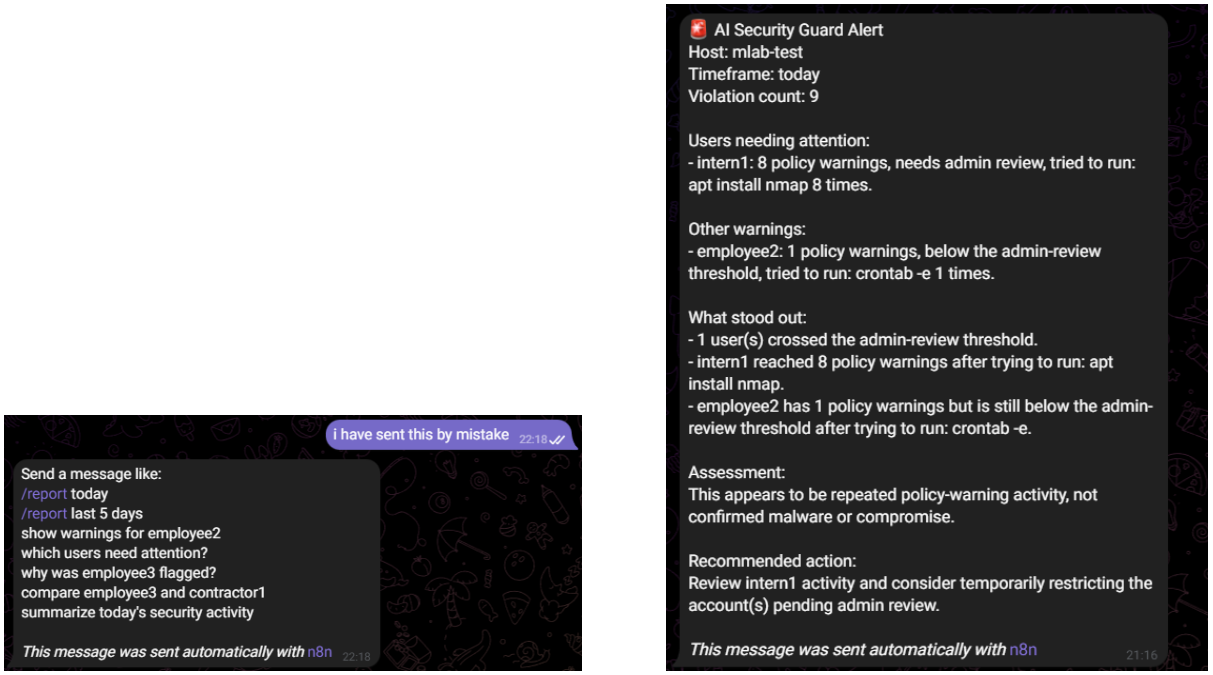
Scenario 1: Clean Security Report

Scenario description. In this scenario, the administrator requested a report for a recent timeframe with no critical activity.

Test input. The example Telegram message was "give me a report for last 1 hour".

Expected behaviour. The workflow should receive the Telegram request, extract the timeframe as last 1 hour, collect logs from `mlab-test`, parse the structured violation summary, generate a plain-English AI security report, and return it to Telegram.

Observed result. This scenario passes if the system correctly returns a clean or low-risk report without inventing violations.



(a) Report request examples (b) AI-generated report

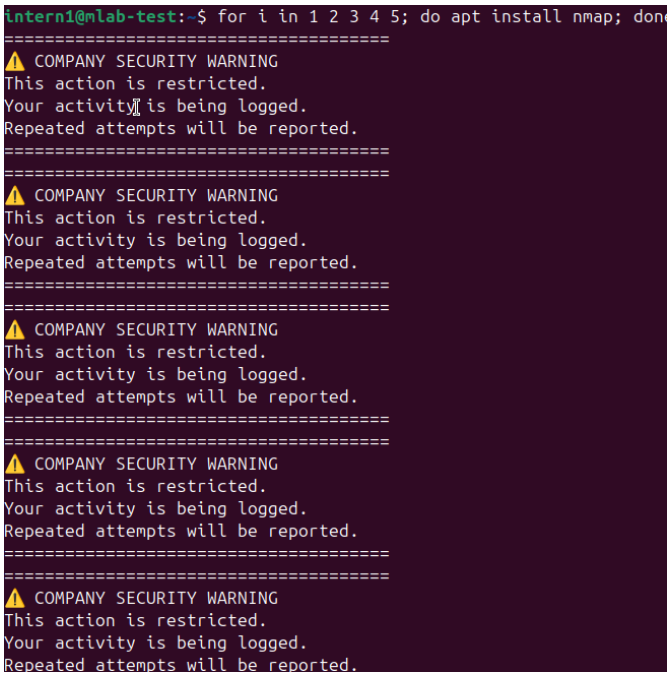
Figure 5.11: Telegram command examples and generated AI Security Guard report for the last 1 hour.

Scenario 2: Policy-Warning Threshold Alert

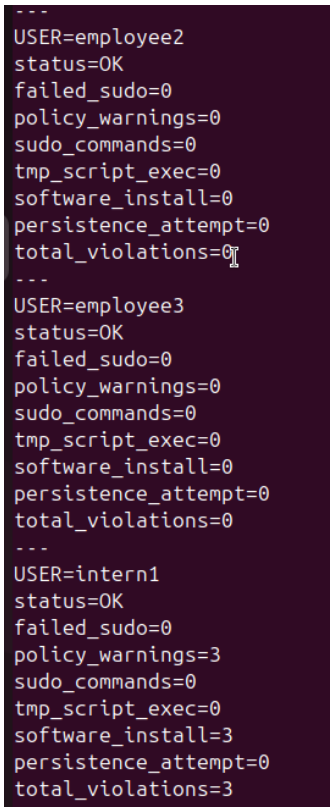
Scenario description. This scenario evaluates whether the scheduled monitoring branch sends an alert only when a user crosses the configured policy-warning threshold.

Expected behaviour. The scheduled trigger should start the workflow, set the timeframe to last 1 hour, retrieve Ubuntu logs through SSH, parse violation counts, detect whether an alert is needed, build the scheduled Telegram alert, and send it to the administrator.

Observed result. The scenario passes if a Telegram alert is sent only after a user reaches five or more policy warnings.



(a) Repeated restricted command attempts



(b) Structured violation summary

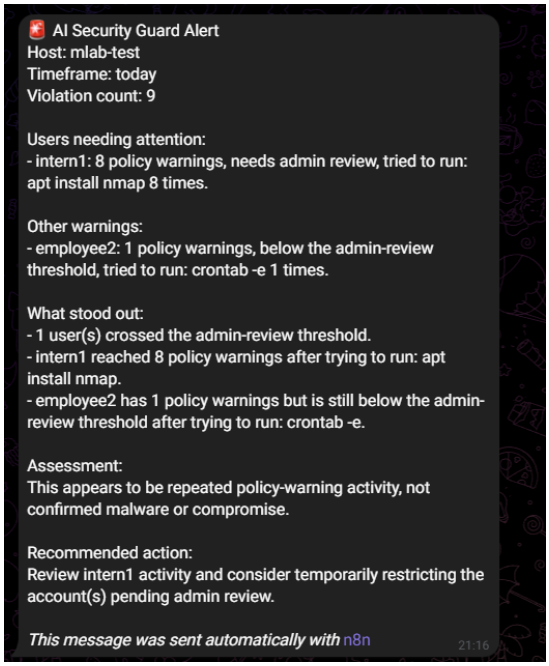
Figure 5.12: Intern user triggering repeated company security warnings and the resulting policy violation summary.

Scenario 3: Timeframe-Based Collection

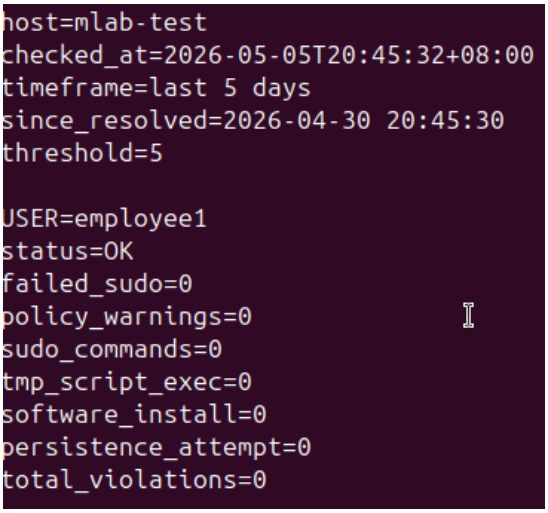
Scenario description. This scenario evaluates whether the collector understands the requested timeframe and applies it when preparing the report.

Expected behaviour. The collector output should include metadata such as host, checked time, requested timeframe, resolved start time, and threshold.

Observed result. The extracted evidence shows timeframe metadata from the collector script, demonstrating that the workflow can evaluate activity within a selected reporting window.



(a) Scheduled Telegram alert



(b) Collector timeframe metadata

Figure 5.13: Scheduled threshold alert and timeframe metadata evidence from the AI Security Guard workflow.

Scenario 4: Normal Chat Message

Scenario description. In this scenario, the administrator sent a message that was not a security report request.

Expected behaviour. The workflow should avoid unnecessary SSH log collection for unrelated Telegram messages.

Observed result. This scenario passes if unrelated messages are not escalated into security log collection workflows.

Summary of Security Findings

The security monitoring workflow demonstrates that a company-managed Ubuntu endpoint can be monitored using Linux logs, audit rules, custom warning scripts, n8n automation,

and AI analysis. The scheduled branch provides automatic alerting when a user reaches the policy-warning threshold, while the Telegram branch allows an administrator to request reports using natural language.

The workflow successfully supports automated log collection from the Ubuntu endpoint, policy-warning detection, admin-review threshold detection, timeframe-based reports, scheduled chatbot alerts, user-specific security reports, and AI-generated plain-English explanations.

The main strength of the workflow is that it does not rely on AI to blindly decide what is dangerous. Linux logs and deterministic rules detect the actual behavior first. The AI layer then explains the evidence and recommends the next action.

Current Limitation and Future Enhancement

One current limitation of the security workflow is that the system mainly monitors command-based and log-based activity. It can detect repeated software installation attempts, policy warnings, and some administrative actions, but it does not yet provide full visibility into all graphical applications or detailed user activity patterns.

A useful future improvement would be to add process accounting or application usage tracking so the system can report which applications users opened during the selected timeframe. This would make the AI Security Guard more practical for real workplace monitoring because it would focus on recent activity, avoid repeated alerts, and provide richer context about user behavior.

5.2 Network Layer Evaluation

This section presents the network-layer evaluation results. The network layer contains two related categories of testing. The first category evaluates the employee-facing issue reporting and policy validation workflow. The second category evaluates the administrator-facing network operations workflow. Together, these workflows demonstrate how the proposed system supports both sides of a network support process: employees can report access problems through Telegram, while administrators can review cases, create diagnostic plans, manage VLAN operations, inspect denied firewall logs, and close cases.

5.2.1 Employee Network Issue Reporting and Policy Validation

Introduction

The employee-side network workflow provides a first point of contact for users who experience network or access-related problems. Instead of immediately escalating every complaint to the network administrator, the workflow first checks whether the problem is already explained by existing firewall or router ACL policy. If the current policy clearly explains the issue, the employee receives a direct plain-English response. If the issue is

not explained by policy, the workflow collects the employee's details and logs a structured case for administrator review.

This design reduces unnecessary support work, improves consistency in handling repeated access complaints, and ensures that unresolved problems are passed to the administrator workflow with enough information for diagnosis.

Workflow Operation

The workflow begins when an employee sends a natural-language Telegram message. The Telegram Trigger receives the message, and the Extract Telegram Message node prepares the chat ID, user information, message content, and timestamp for processing. The AI Agent then classifies the message into one of four categories: general chat, policy check, network issue, or clarification needed.

For policy-related messages, the workflow calls two policy evidence sources: the firewall rules workflow and the router ACL list workflow. The results are merged and interpreted by the policy snapshot logic. If the issue is clearly restricted by firewall or ACL policy, the workflow sends a direct response and does not create a case. If policy evidence does not explain the problem, the workflow asks for the employee's full name, employee number, email address, department, and issue description. Once complete, it creates a structured support case.

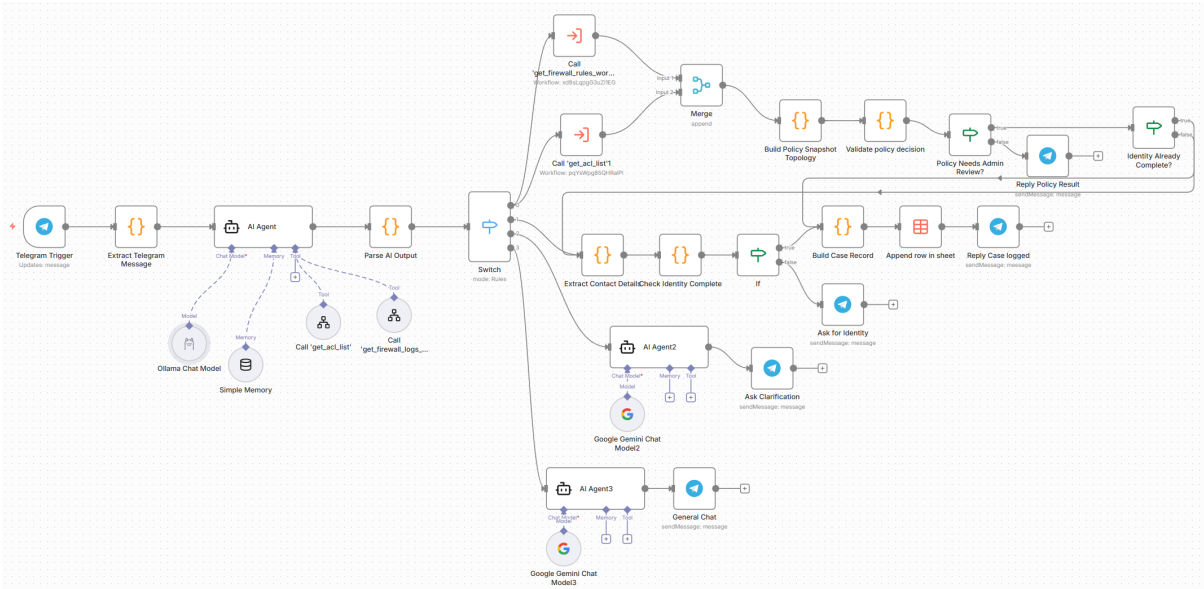


Figure 5.14: Employee Network Case Logging workflow design in n8n.

Figure 5.14 shows the actual n8n implementation of the employee-side workflow. The workflow starts from the Telegram Trigger, prepares the message using the Extract Telegram Message node, classifies the request using the main AI Agent, parses the AI output, and routes the result through the Switch node. The branches support policy validation, network issue case logging, clarification requests, and general chat responses.

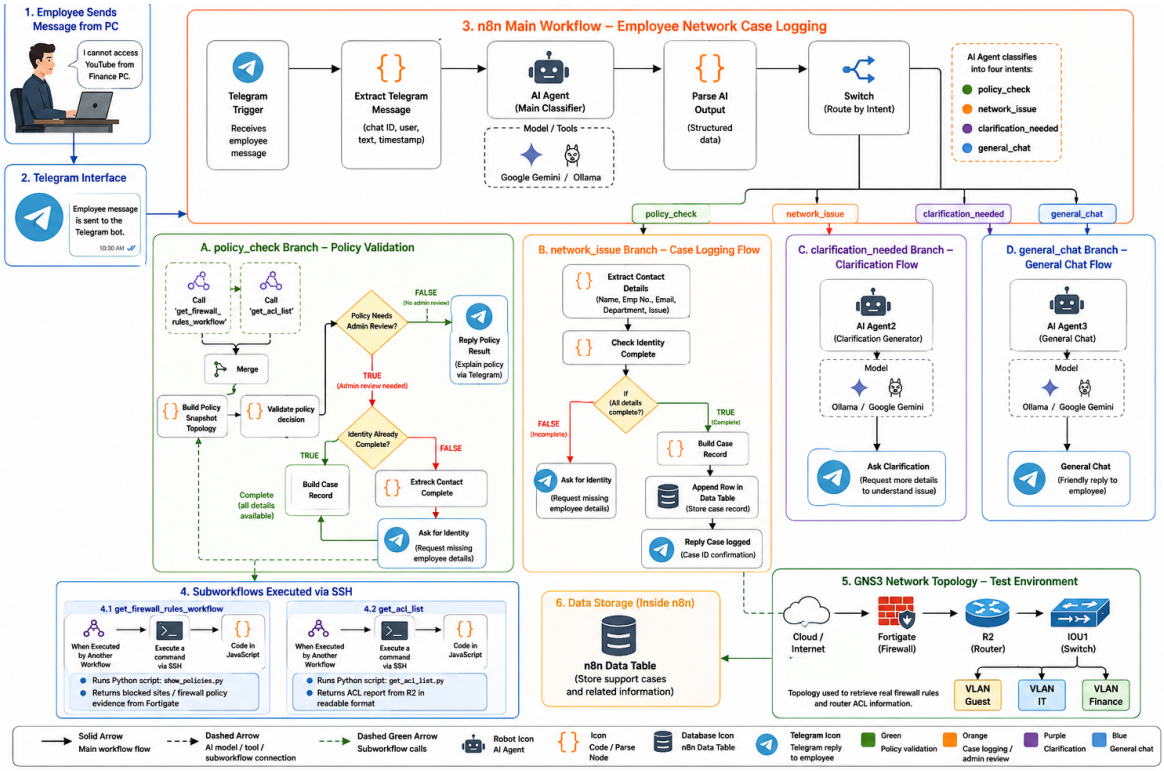


Figure 5.15: General operational flow of the Employee Network Issue Reporting and Policy Validation workflow.

Figure 5.15 summarizes the workflow from employee message intake to final response. The employee message enters through Telegram and is classified by the main AI Agent into policy check, network issue, clarification needed, or general chat. Policy checks use firewall and ACL sub-workflows executed through SSH to collect evidence from the Fortigate firewall and router ACL sources in the GNS3 test topology. Network issues are routed to contact-detail extraction, identity completeness checks, case record creation, and n8n data-table storage. The diagram shows that AI is used for classification, while n8n branches, policy sub-workflows, Python scripts, and stored case records provide controlled execution and traceable support handling.

Testing Scenarios and Results

The employee-side tests focused on whether the workflow could distinguish between policy-explained problems and real support cases requiring administrator review.

Scenario 1: Known Blocked External Platform

Test input. The employee sent: "I'm using the Finance pc and I cannot access YouTube".

Observed result. The workflow identified YouTube as blocked by the company's internet access policy and returned a direct explanation to the employee. No support case was logged because the issue was already explained by current firewall policy.

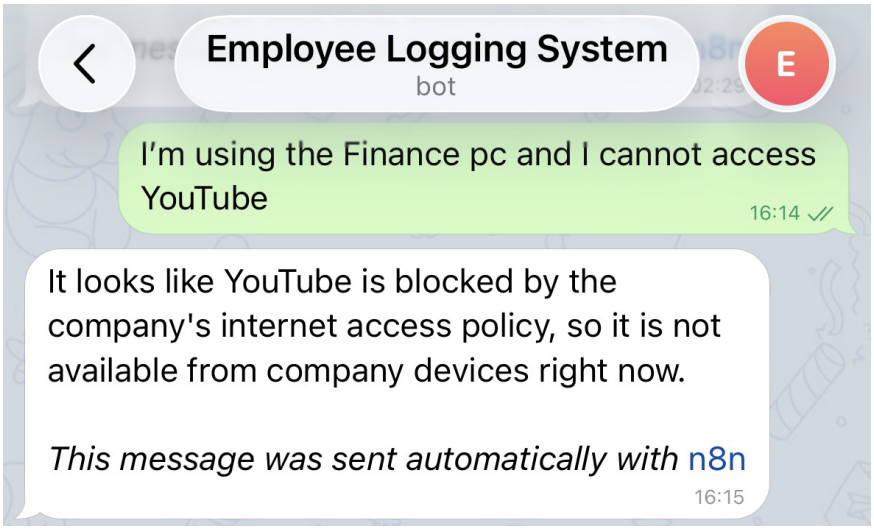


Figure 5.16: Employee-side policy validation for blocked YouTube access.

Figure 5.16 shows a policy-explained external access problem. The employee reports that YouTube cannot be accessed from a Finance PC, and the workflow replies that YouTube is blocked by the company's internet access policy. This scenario demonstrates that the employee-side bot can answer known policy restrictions directly without creating an unnecessary support case.

Scenario 2: Guest Network Attempting Internal Access

Test input. The employee sent: "I'm using the guest pc and I cannot reach the HR department".

Observed result. The workflow identified that the request originated from the Guest network and matched it to router ACL restrictions that prevent guest users from accessing internal departments. It returned a direct explanation and did not create an unnecessary case.

Figure 5.17 shows an internal access request that is explained by router ACL policy. The employee is using the Guest network and cannot reach the HR department. The workflow identifies this as a policy restriction because guest users are not allowed to access internal company departments. The bot therefore explains the restriction directly instead of escalating it as a technical fault.

Scenario 3: Internal Department Issue Requiring Case Logging

Test input. The employee sent: "I'm using the Finance pc and I cannot reach the HR department".

Observed result. The workflow did not treat this as a simple guest-network policy restriction. It requested the employee's identity and issue details, then logged a support case after the required information was provided.

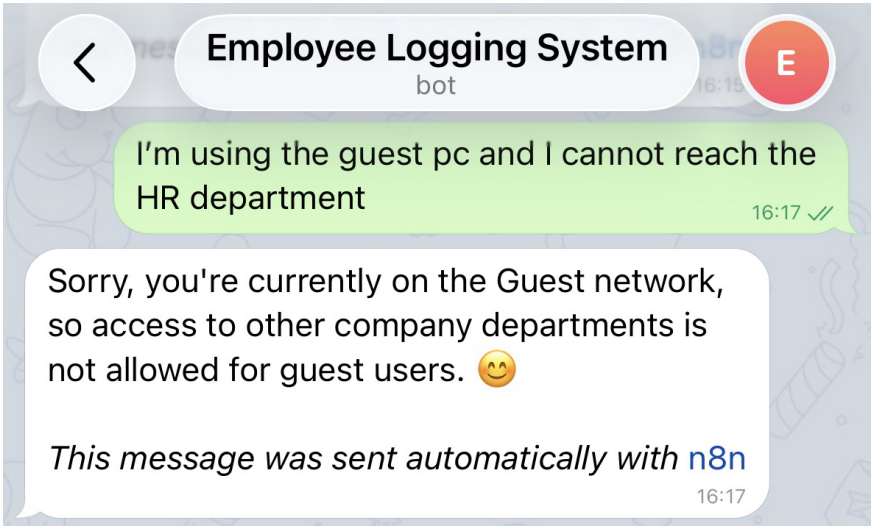


Figure 5.17: Employee-side ACL validation for Guest network access to the HR department.

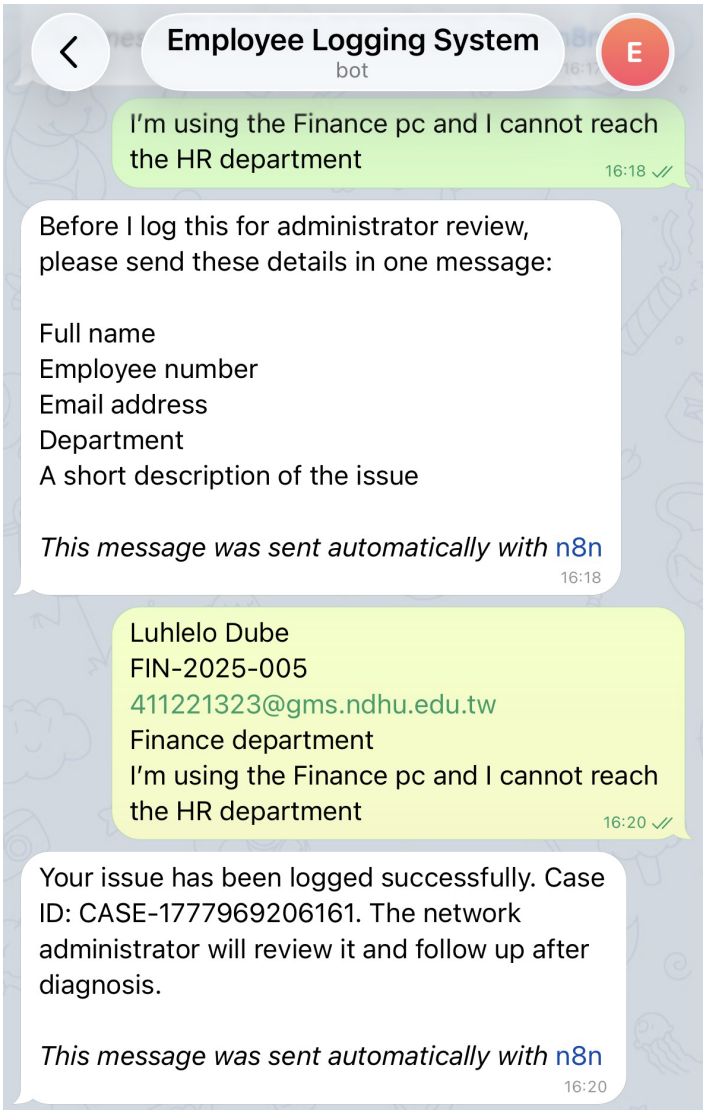


Figure 5.18: Employee-side case logging for an unresolved internal department access issue.

Figure 5.18 shows the case-logging path for an issue that is not clearly explained by existing policy. The Finance-to-HR access problem is treated as requiring administrator review. The workflow asks for missing employee identity details, receives the completed information, logs the issue, and returns a case ID. This demonstrates escalation from chatbot intake to structured support-case creation.

Scenario 4: Internal Server Access Issue Requiring Case Logging

Test input. The employee sent: "I cannot access Ubuntu server from the IT department".

Observed result. The workflow requested the missing employee details, received the completed form, created a case record, and returned a case ID for administrator follow-up.

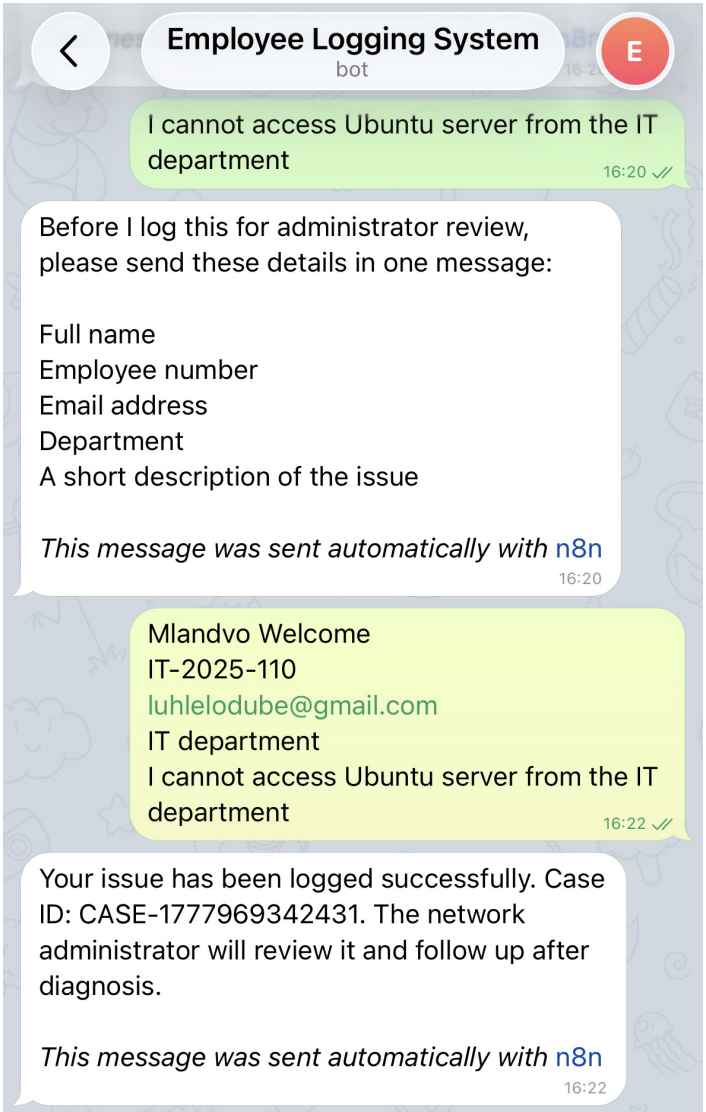


Figure 5.19: Employee-side escalation and case logging for an internal server access issue.

Figure 5.19 shows a second unresolved issue being escalated for administrator follow-up. The employee reports that an Ubuntu server cannot be accessed from the IT

department. Because the workflow cannot resolve the issue as a simple firewall or ACL policy explanation, it collects the required employee details, creates a structured case record, and returns the case ID. This confirms that internal server access problems can be routed into the administrator support queue.

Employee Workflow Findings

The employee-side evaluation shows that the workflow can reduce unnecessary escalation by explaining known policy restrictions directly. It also shows that unresolved network problems can be converted into structured support cases. The main limitation is that the workflow does not yet assign urgency based on language such as "urgent", "critical", or "as soon as possible". Adding urgency detection would help prioritize high-impact incidents. Another limitation is that the workflow depends on the availability and accuracy of the firewall and ACL sub-workflows. If those evidence sources fail or return incomplete information, the safest action is to escalate the issue for administrator review.

5.2.2 Admin Network Operations Bot Evaluation

Introduction

The Admin Network Operations Bot is the administrator-side workflow for network support and operational control. It is implemented in n8n and connected to Telegram. Its purpose is to give the network administrator a centralized interface for reviewing employee support cases, generating diagnostic plans, managing VLAN operations, retrieving denied firewall logs, and notifying employees after a case is resolved.

Figure 5.20 summarizes the main administrator workflow from Telegram request intake to final response delivery. The workflow receives the message, extracts key fields, checks whether the request belongs to an existing approval process, and then routes non-approval requests through AI interpretation and validation. Depending on the interpreted intent, the request is directed to case management, diagnostic planning, VLAN operations, denied firewall log retrieval, or fallback support. The result is returned through Telegram or, for resolved cases, through Gmail.

Evaluation Environment and Supported Inputs

The workflow was evaluated as a Telegram-based n8n network administration assistant. It supports command-style and natural-language requests such as:

- `show open cases`
- `show cases Finance`
- `view CASE-1777527657268`

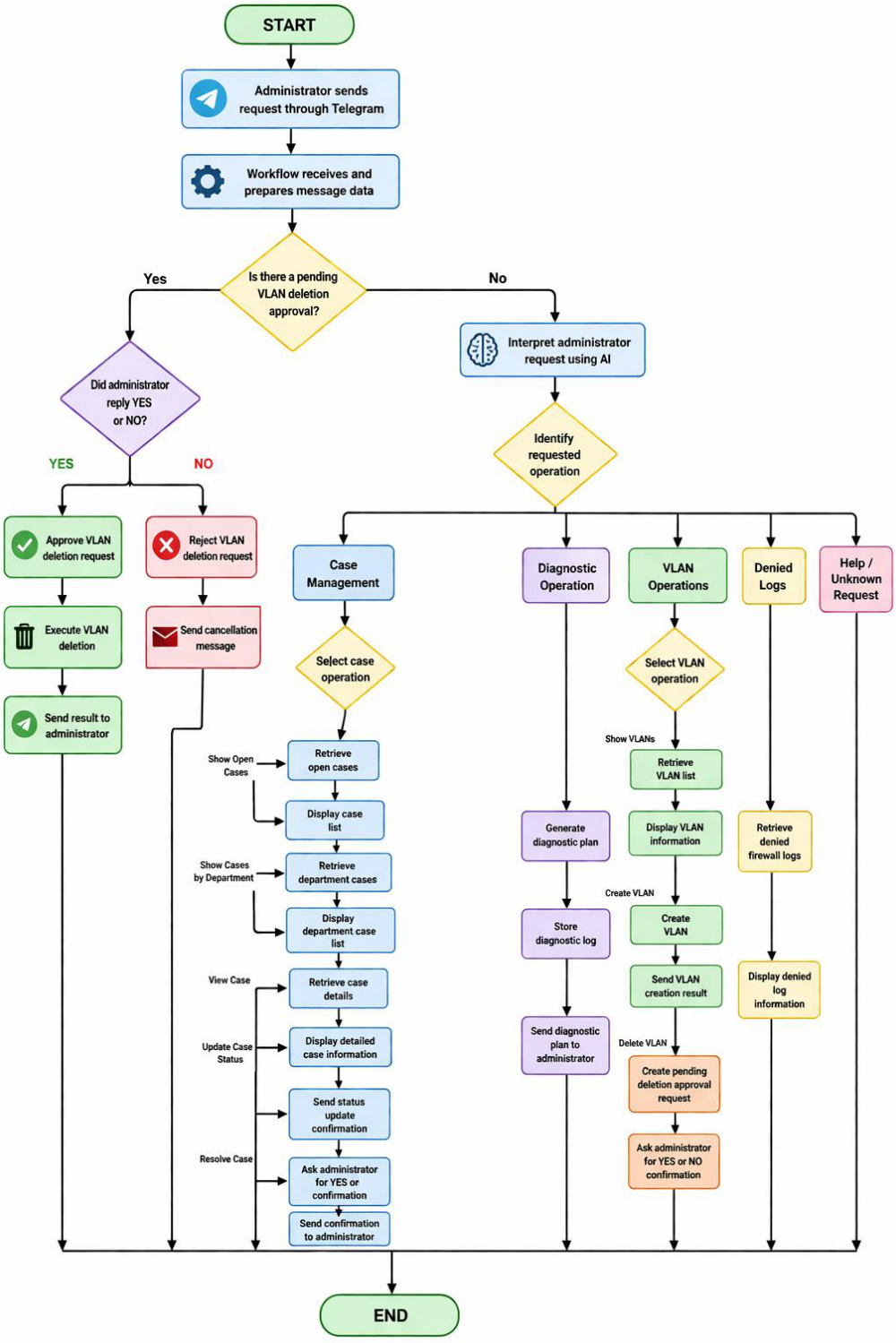


Figure 5.20: General operational flow of the Admin Network Operations Bot workflow.

- `diagnose Bruno Maseko`
- `update Bruno Maseko status In Progress`
- `resolve Bruno Maseko restored access`
- `show vlans`
- `create vlan 120 HR_USERS`
- `delete vlan 120`
- `get denied logs`

The help output confirms that the bot presents these supported network operations directly to the administrator.

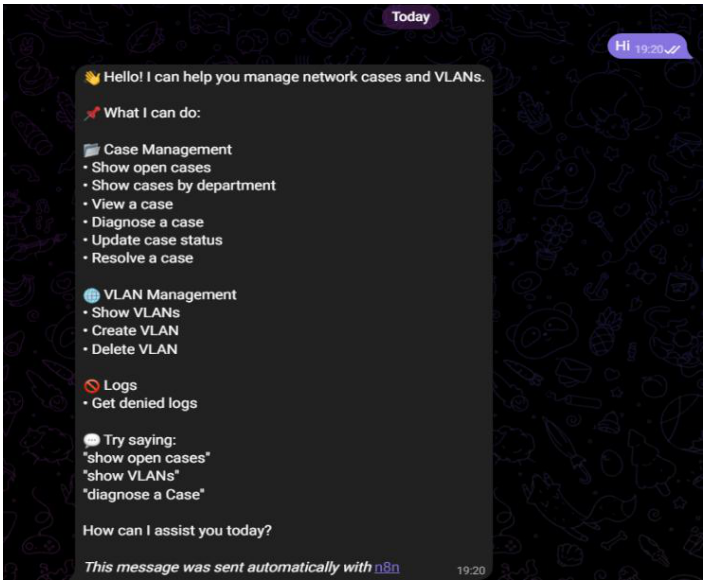


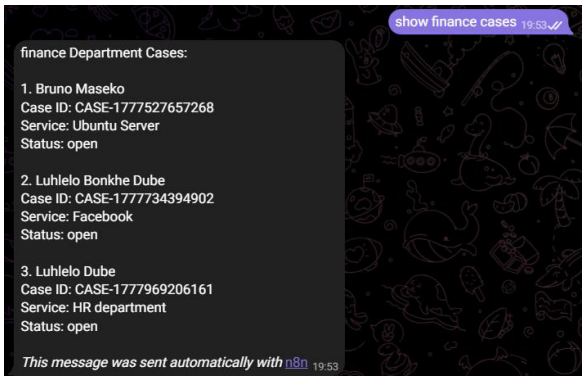
Figure 5.21: Admin Network Operations Bot help output.

Figure 5.21 shows the help and fallback behavior of the administrator bot. When a request is unclear, unsupported, or incomplete, the workflow does not fail silently. Instead, it returns guidance showing the supported operations. This improves usability because the administrator can recover quickly from incorrect wording and continue with a valid command.

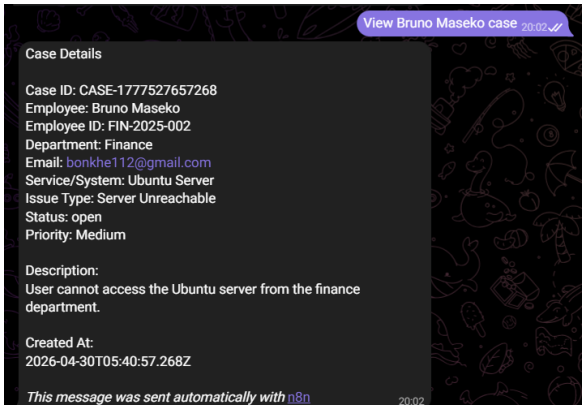
Case Management Results

The case management branch allows the administrator to inspect support records created by the employee-side workflow. The tested operations include department filtering, viewing full case details, updating status, resolving a case, and sending a resolution email.

Figure 5.22 shows how the administrator can inspect support cases from Telegram. The department-filtered view narrows the results to cases connected to a selected unit,



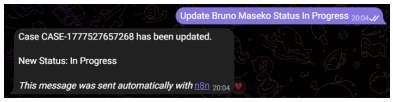
(a) Department case filtering



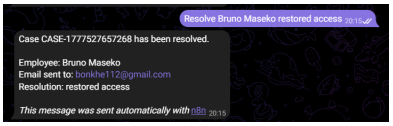
(b) Detailed case view

Figure 5.22: Administrator case inspection through Telegram.

such as Finance, which helps identify localized problems and repeated issues. The detailed case view then retrieves the full record for one support case, including the case identifier, employee details, department, issue description, priority, and status. This gives the administrator enough context before diagnosing, updating, or resolving the case.



(a) Status update



(b) Case resolution



(c) Employee email

Figure 5.23: Case status update, resolution confirmation, and employee notification.

Figure 5.23 demonstrates the case lifecycle after inspection. The status update confirms that the stored case record can be changed, for example from open to in progress. The resolution confirmation shows that the workflow can close the case after the administrator provides the outcome. The email notification proves that the system also communicates the result back to the affected employee, closing the support loop rather than only updating an internal record.

These results show that the administrator workflow completes the support-case lifecycle rather than only displaying reports. It can move a case from review to resolution and communicate the outcome to the affected employee.

Diagnostic Planning Results

The diagnostic planning module generates a structured troubleshooting plan for a selected case. The plan includes reachability checks, VLAN assignment checks, routing or interface review, ACL and firewall inspection, and a likely next step. The output is returned in Telegram and stored in the diagnostic logs table.

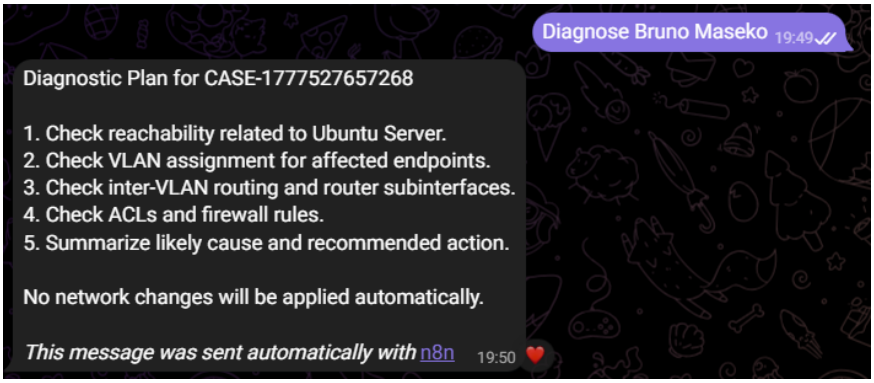
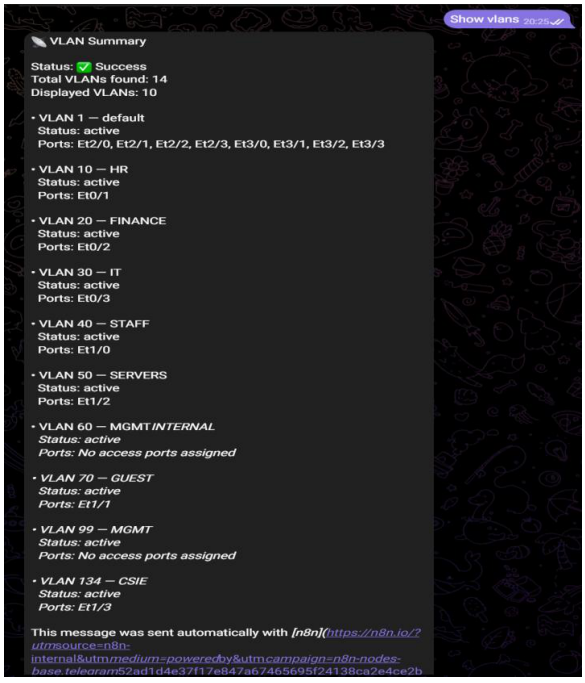


Figure 5.24: Administrator diagnostic plan generated for a network support case.

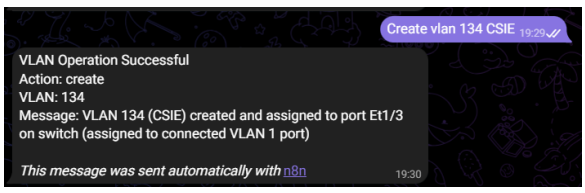
Figure 5.24 shows the structured diagnostic plan generated for a selected support case. The plan guides the administrator through checks such as reachability testing, VLAN assignment verification, routing or subinterface review, ACL inspection, and firewall policy review. This supports consistent troubleshooting without immediately performing live remediation or making automatic network changes.

VLAN Operations and Approval Control

The VLAN branch supports viewing VLANs, creating VLANs, and deleting VLANs. VLAN creation is treated as a routine provisioning action, while VLAN deletion is protected by an approval gate because it can disconnect devices assigned to the VLAN.



(a) VLAN inventory output



(b) VLAN creation result

Figure 5.25: VLAN visibility and VLAN creation results.

Figure 5.25 shows the read and create functions of the VLAN operations branch.

The VLAN inventory output gives the administrator quick visibility into the current segmentation environment without opening separate network tools. The VLAN creation output shows that the workflow can pass a VLAN ID and name to a child workflow for provisioning, demonstrating that the bot can perform controlled infrastructure changes and not only display information.

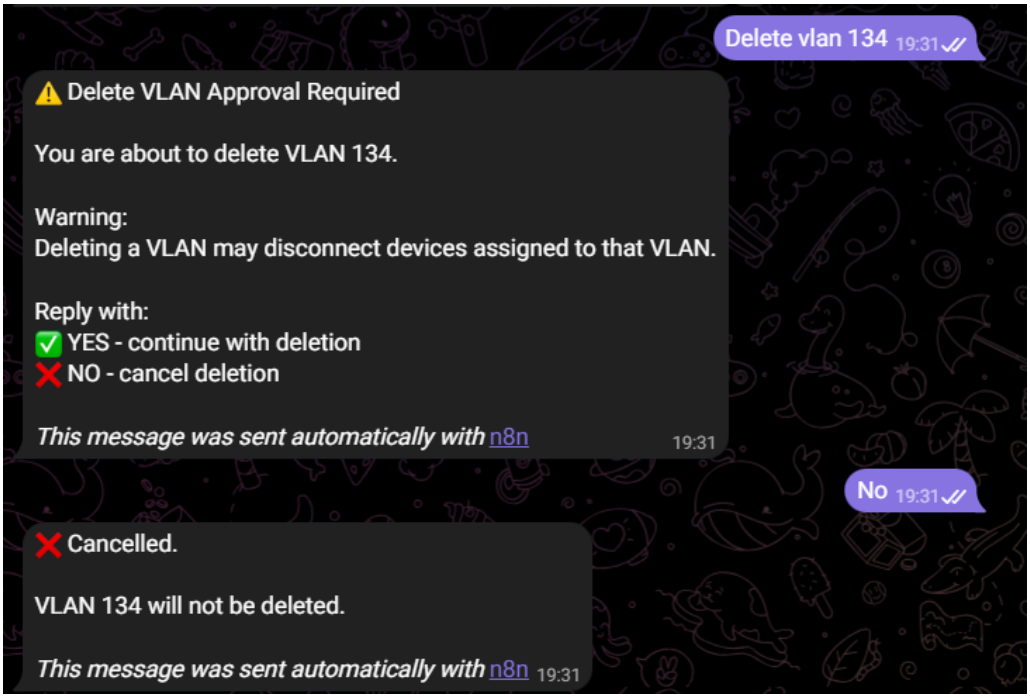


Figure 5.26: Approval-controlled VLAN deletion request and rejection result.

Figure 5.26 shows the approval control applied to VLAN deletion. The workflow does not delete the VLAN immediately; it stores the pending action and asks the administrator to reply with approval or rejection. When the deletion is rejected, the workflow cancels the pending action and prevents the destructive operation from taking place. This confirms a fail-safe design in which disruptive network changes only occur after explicit authorization.

Denied Firewall Log Retrieval

The denied firewall log branch retrieves blocked-traffic evidence from a child workflow. This is useful when access complaints may be caused by intentional security restrictions rather than technical failure.

Figure 5.27 shows denied firewall log information returned by the workflow. This output helps the administrator determine whether an access problem is caused by an intentional security rule rather than a technical fault. By showing blocked traffic evidence with source, destination, VLAN, and service context, the workflow supports faster policy verification and evidence-based troubleshooting.

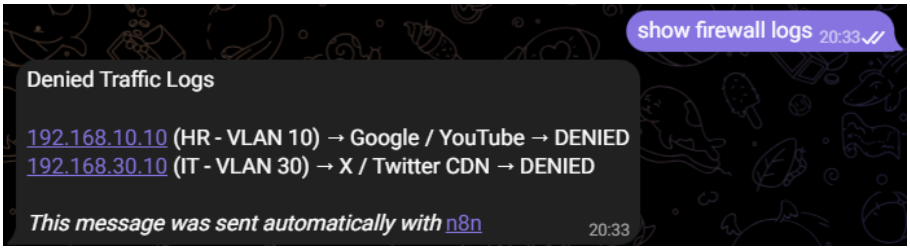
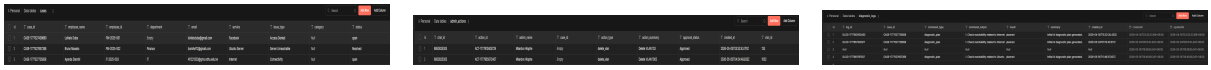


Figure 5.27: Denied firewall log retrieval output.

Data Records and Audit Evidence

The workflow uses internal data tables to preserve operational state. The cases table stores support records, the admin actions table stores protected action requests such as VLAN deletion approval, and the diagnostic logs table stores generated troubleshooting plans.



(a) Cases table (b) Admin actions table (c) Diagnostic logs table

Figure 5.28: Network administration data tables used for support records, approval state, and diagnostic history.

Figure 5.28 shows the persistent data records used by the administrator workflow. The cases table stores employee support records used for viewing, filtering, updating, and resolving issues. The admin actions table stores protected actions such as pending VLAN deletion requests, allowing the workflow to remember approval state across separate messages and preserve an audit trail. The diagnostic logs table stores generated troubleshooting plans for future reference, repeated-issue comparison, and project documentation. These records improve traceability because case changes, approval-controlled actions, and diagnostic guidance are stored instead of existing only as temporary chat messages.

Network Layer Summary

The network-layer evaluation shows that the project supports both employee-facing and administrator-facing network operations. The employee workflow filters policy-explained problems before escalation, while the administrator workflow handles case review, troubleshooting, VLAN operations, firewall evidence retrieval, and employee notification. The strongest finding is that the system does not treat the chatbot as a standalone answer generator. Instead, the chatbot sits on top of deterministic workflow branches, policy evidence, data tables, and approval controls.

Chapter 6

Discussion

6.1 Discussion Overview

This chapter discusses the findings from the experimental evaluation. The evaluation covered two main layers: the systems layer and the network layer. Within the systems layer, the evaluated components were systems infrastructure administration and systems security monitoring. Within the network layer, the evaluated components were employee network issue reporting and administrator network operations.

6.2 Interpretation of Findings

The results show that the proposed system can support practical infrastructure management tasks when AI is combined with structured workflow automation. Across the evaluated components, the AI layer improves usability by allowing users to describe requests in natural language. However, the actual operational behavior remains controlled by n8n routing, deterministic parsing, data tables, workflow branches, SSH/API calls, and explicit approval gates.

In the systems administration evaluation, the Proxmox Admin Bot successfully supported safe patching, resource diagnostics, unreachable VM triage, idle VM cleanup recommendations, and IP discovery. These results demonstrate that a chat-based interface can reduce the manual effort required to gather infrastructure evidence. The strongest systems administration result was the safe patching workflow, because it combined human approval, snapshot creation, update execution, health checks, and rollback guidance.

The systems security monitoring evaluation showed that security automation benefits from separating evidence collection from AI explanation. Linux logs, audit rules, and deterministic threshold logic identify policy-warning activity. The AI layer then explains the evidence in plain English. This reduces the risk of allowing the AI model to invent or independently decide security violations.

The network employee workflow showed that first-level support can be improved by checking policy before escalation. Known blocked services and Guest network restrictions were explained directly to the employee. When the problem was not clearly explained by firewall or ACL policy, the workflow collected identity details and created a structured case. This confirms the usefulness of policy-aware triage.

The administrator network workflow demonstrated the other side of the support lifecycle. The administrator could inspect cases, update status, resolve cases, generate diagnostic plans, create VLANs, retrieve denied firewall logs, and control VLAN deletion

through approval. This shows that the project supports both intake and operational follow-up rather than only producing chatbot responses.

6.3 Human-in-the-Loop Control

A repeated theme across the evaluation is the importance of human-in-the-loop control. Risky operations were not treated the same as read-only diagnostics. Systems patching required approval before execution. VM reboot, stop, rollback, and destructive maintenance are treated as approval-gated actions. In the network layer, VLAN deletion required explicit confirmation and could be rejected.

This control pattern is important because infrastructure automation can cause service disruption if it acts too aggressively. The evaluation suggests that AI-assisted infrastructure administration should not be fully autonomous by default. A safer model is to allow AI to interpret, summarize, and recommend while deterministic workflow logic enforces boundaries around execution.

6.4 Operational Benefits

The implemented workflows provide several operational benefits:

- faster access to infrastructure evidence through Telegram;
- reduced manual checking across Proxmox, Linux, firewall, ACL, and case records;
- consistent formatting of diagnostic and support information;
- direct policy explanations for repeated employee access complaints;
- structured case records for unresolved issues;
- approval gates before risky actions;
- data-table records for cases, admin actions, and diagnostic logs.

These benefits are especially relevant in small or educational infrastructure environments where administrators may not have access to enterprise-grade monitoring suites but still need consistent operational support.

6.5 Limitations

The prototype has several limitations. First, the system depends on Telegram, n8n, SSH/API connectivity, and supporting child workflows. If any of these services fail, the automation path may fail even if the underlying infrastructure is still operational.

Second, natural-language interpretation can still be ambiguous. The project reduces this risk through deterministic routing and validation, but unclear user messages may still require clarification or may be classified incorrectly.

Third, the systems security workflow mainly observes command-based and log-based activity. It does not yet provide full application usage tracking or complete visibility into graphical user behavior.

Fourth, the employee network workflow does not yet perform urgency detection. A routine access request and a high-impact urgent outage may be logged similarly unless priority is added manually or through future logic.

Finally, the prototype was evaluated in a controlled laboratory environment. This improves repeatability but means that production concerns such as large-scale credential rotation, high availability, role-based access control, audit retention, and enterprise monitoring integration remain future deployment requirements.

6.6 Lessons Learned

The main lesson from the evaluation is that AI-assisted infrastructure management works best when AI is placed inside a controlled automation architecture. The useful pattern is not to let the model directly manage infrastructure. Instead, the model helps interpret natural language and explain results, while workflows enforce validation, approval, logging, and separation between read-only diagnostics and write operations.

Another lesson is that modular workflow design improves maintainability. The project separates functions into specialist workflows such as IP resolution, resource diagnostics, snapshot management, safe patching, policy validation, VLAN operations, denied log retrieval, and case management. This makes each workflow easier to test and reduces the risk that one failure affects the entire system.

The evaluation also shows that chat interfaces are most useful when the output is operationally structured. The best results were not generic chatbot answers, but formatted outputs containing VMIDs, IP addresses, case IDs, policy reasons, snapshot names, resource metrics, and recommended next steps.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project designed, implemented, and evaluated an AI-assisted workflow automation system for infrastructure management. The system combined natural-language interaction, n8n workflow orchestration, Telegram-based user interfaces, Proxmox and Linux automation, network policy validation, VLAN operations, case management, and human-in-the-loop approval controls.

The evaluation showed that the system can assist with both systems-layer and network-layer operations. At the systems layer, the prototype supported safe VM patching, resource diagnostics, unreachable VM triage, idle VM cleanup recommendations, IP discovery, and security monitoring. At the network layer, the prototype supported employee issue reporting, policy-aware access explanations, structured case logging, administrator case review, diagnostic planning, VLAN operations, firewall log retrieval, and employee notification after case resolution.

The main contribution of the project is the demonstration that AI can be useful in infrastructure administration when it is combined with deterministic automation and operational controls. The AI component improves interaction and explanation, but the workflows preserve structure, traceability, and safety. This is especially important for infrastructure tasks, where incorrect automation can disrupt services.

Overall, the project supports the argument that AI-assisted network and systems management is most practical when implemented as a supervised automation architecture rather than as an unrestricted autonomous agent.

7.2 Project Contributions

The project makes the following practical contributions:

- a Telegram-based interface for systems and network administration tasks;
- modular n8n workflows for infrastructure operations;
- human approval gates for risky systems and network actions;
- Proxmox-based systems administration automation;
- Linux endpoint security monitoring with AI-generated explanations;

- employee-facing network policy validation and case logging;
- administrator-facing network case management and VLAN operations;
- experimental evidence showing successful workflow execution across multiple scenarios.

7.3 Future Work

Several improvements would make the system stronger and more suitable for production-like environments.

First, the employee network workflow should include urgency detection. Messages containing terms such as “urgent”, “critical”, or “business outage” should be assigned higher priority and surfaced more quickly to the administrator.

Second, the systems security workflow should be expanded with application usage or process accounting. This would allow the system to report not only command-line policy violations but also broader endpoint behavior during selected timeframes.

Third, role-based access control should be added to the Telegram interfaces. Different users should have different permissions, especially for destructive actions such as VM rollback, reboot, VLAN deletion, and infrastructure changes.

Fourth, production credential management should be improved through a dedicated secrets backend such as HashiCorp Vault or an equivalent platform. This would support credential rotation and reduce the risk of exposing SSH keys, API tokens, or service credentials.

Fifth, the workflows should be connected to broader observability tools such as centralized logging, metrics dashboards, and alerting systems. This would allow the chatbot to use richer evidence and support more advanced incident response.

Finally, future work should evaluate the system with more users, more devices, more failure conditions, and longer-running operational tests. This would help measure reliability, response time, classification accuracy, and administrator trust over time.

7.4 Final Remarks

The completed prototype demonstrates that AI-assisted infrastructure automation can reduce manual effort, improve consistency, and preserve safety when implemented with clear workflow boundaries. The project does not replace the administrator. Instead, it gives the administrator a more efficient interface for evidence gathering, triage, policy validation, and controlled execution.

Bibliography

[1] Amar Abane, Abdella Battou, and Mheni Merzouki. An adaptable ai assistant for network management. In *Conference manuscript / demo paper from provided PDF*, 2024. Bibliographic venue details were incomplete in the provided file; cite with caution and verify before final submission.

[2] Moussa Aboubakar, Mounir Kellil, and Pierre Roux. A review of iot network management: Current status and perspectives. *Journal of King Saud University – Computer and Information Sciences*, 34:4163–4176, 2022.

[3] Cisco. Model driven telemetry white paper, 2024. Vendor white paper; not peer reviewed.

[4] Pragati Das. Ai-based network management for iot devices. *Computational Engineering and Technology Innovations*, 1(1):44–51, 2024.

[5] Yuyang Du, Qun Yang, Liujianfu Wang, Jingqi Lin, Hongwei Cui, and Soung Chang Liew. Llmind 2.0: Distributed iot automation with natural language m2m communication and lightweight llm agents. *arXiv preprint arXiv:2508.13920*, 2025. Preprint.

[6] Mohd Faris Mohd Fuzi, Khairunnisa Abdullah, Iman Hazwam Abd Halim, and Rafiza Ruslan. Network automation using ansible for eigrp network. *Journal of Computing Research and Innovation*, 6(4), 2021.

[7] Jibum Hong, Nguyen Van Tu, and James Won-Ki Hong. A comprehensive survey on llm-based network management and operations. *International Journal of Network Management*, 35:e70029, 2025.

[8] Santhosh K and Gary D. Self-healing networks: Implementing ai-powered mechanisms to automatically detect and resolve network issues with minimal human intervention, 2023. Use with caution; venue rigor and metadata quality appear weak in the provided file.

[9] Diego Kreutz, Fernando M. V. Ramos, Paulo Verissimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1):14–76, 2015. Provided source file was arXiv v1 (2014); canonical journal version commonly cited as 2015.

[10] Omondi James Okeda. Ai-driven network management and optimization: An exploratory literature review, 2025. ScienceOpen preprint; not peer reviewed.